

PYTHON SCIENTIFIC COMPUTING

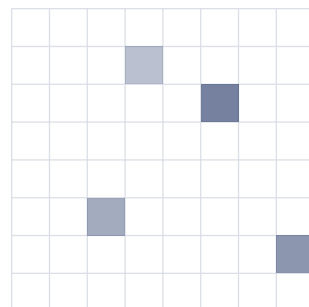
Python 科学计算

课程笔记

Scientific Computing with Python Lecture Notes

作者：隴千夏

日期：2026 年 5 月 6 日



目录

讲义结构	1
1 科学计算与 Python 编程基础	2
1.1 课程定位：用 Python 组织科学计算流程	2
1.2 开发环境：从解释器到 Notebook	2
1.2.1 环境启动与检查	3
1.2.2 Notebook 的基本工作流	4
1.3 Python 如何组织代码	4
1.3.1 变量与基本类型	4
1.3.2 分支：让程序根据条件选择路径	5
1.3.3 循环：重复执行同一类操作	6
1.3.4 函数：把可复用逻辑命名	7
1.3.5 模块、包与导入	8
1.3.6 类：把数据和行为放在一起	8
1.4 Python 如何组织数据	9
1.4.1 序列：字符串、列表与元组	9
1.4.2 字典：用键组织信息	10
1.4.3 集合：关心成员而非顺序	11
1.4.4 嵌套结构：从树状草图到复杂数据	12
1.4.5 递归、时间开销与数据结构选择	12
1.4.6 选择数据结构的原则	13
1.5 复习与练习	14
2 NumPy 数组计算	15
2.1 为什么科学计算需要 NumPy	15
2.2 ndarray: NumPy 的主要数组类型	16
2.2.1 数组的数据类型	16
2.3 数组属性：先看清数据的结构	17
2.4 实验上：数组结构检查与基础访问	18
2.5 数组生成：从手写数据到规则数据	19
2.5.1 由已有数据创建数组	19
2.5.2 零数组、一数组和指定值数组	20
2.5.3 等差序列与线性空间	20
2.5.4 单位矩阵和随机数组	21
2.6 基本操作：索引、切片、变形与转置	21
2.6.1 一维数组的索引和切片	21
2.6.2 二维数组的索引和切片	22
2.6.3 变形：reshape	22
2.6.4 展平、转置和复制	23
2.6.5 插入、追加和删除	23

2.7	数组合并与分割	24
2.7.1	合并数组	24
2.7.2	分割数组	25
2.8	实验中：从网页或文件得到表格型数据	25
2.9	数组运算：向量化与广播	27
2.9.1	数组与标量运算	27
2.9.2	数组与数组运算	28
2.9.3	通用函数	28
2.9.4	广播	28
2.10	聚合运算：把数组压缩为统计量	29
2.11	arg 索引运算：找位置而不只是找值	30
2.12	比较、布尔索引与 Fancy Index	32
2.12.1	数组比较	32
2.12.2	布尔索引	32
2.12.3	Fancy Index	33
2.13	实验一：用 NumPy 支撑数据可视化	34
2.14	实验二：用 NumPy 实现 KNN 的主要步骤	35
2.15	复习与练习	38
3	SciPy 科学计算	39
3.1	SciPy 的模块结构	39
3.2	常数包和特殊函数	39
3.3	优化：最小二乘拟合、函数最小值和方程求解	41
3.4	实验：最小二乘法和梯度下降	43
3.5	插值计算	44
3.6	线性代数	45
3.7	数值积分和常微分方程	46
3.8	实验：图像数组与 SciPy 处理流程	47
3.9	信号处理	48
3.10	傅里叶变换	49
3.11	实验：信号滤波和频域分析	50
3.12	统计分布与假设检验	50
3.13	复习与练习	52
4	Matplotlib 绘图与可视化	54
4.1	绘图环境与基本工作流	54
4.2	折线图：从曲线到可读图形	55
4.3	图形对象、子图和布局	56
4.4	散点图与分类数据	57
4.5	常用统计图形	58
4.5.1	柱状图和水平柱状图	58
4.5.2	直方图和二维图像	59
4.5.3	饼图、堆叠面积图和箱线图	60

4.6	标注、图例与图像导出	60
4.7	动画可视化	62
4.8	实验：排序过程的动画表示	64
4.9	图像像素与三维散点图案例	65
4.10	复习与练习	67
5	Pandas 数据分析	69
5.1	从表格读取开始	69
5.2	Series：带索引的一维数据	70
5.3	DataFrame：二维表格对象	71
5.4	索引、切片与布尔筛选	72
5.5	缺失值、重复值与基础预处理	72
5.6	apply：把自定义规则作用到行或列	73
5.7	合并表格：merge、concat 和 combine	74
5.8	分箱、分组与聚合	75
5.9	透视表与宽长表转换	77
5.10	实验：Iris 数据预处理与相关性分析	78
5.11	实验：泰坦尼克号数据分析	79
5.12	复习与练习	81
6	OpenCV 图像处理与计算机视觉	83
6.1	图像显示、颜色通道与像素	83
6.2	灰度图、直方图与阈值分割	84
6.3	轮廓、外接框与形态学	85
6.4	实验：手写数字图像处理	87
6.5	实验：手写数字识别	88
6.6	图像滤波	89
6.7	图像边缘检测	91
6.8	实验：车道线滤波检测	92
6.9	图像几何变换	93
6.10	复习与练习	95
7	课程综合项目	96
7.1	项目一：表格数据分析报告	96
7.2	项目二：带噪声信号的频域分析	97
7.3	项目三：图像目标提取	98
7.4	综合项目复习要求	99

讲义结构

节次	主题	内容定位
第 1 节	科学计算与编程基础	课程介绍、环境安装、Python 代码组织、Python 数据组织
第 2 节	NumPy 数组计算	数组类型、数组生成、索引、运算、聚合和数组实验
第 3 节	SciPy 科学计算	优化、插值、线性代数、积分、信号处理和傅里叶变换
第 4 节	Matplotlib 可视化	基础绘图、常用图形、动画和三维绘图案例
第 5 节	Pandas 数据分析	Series、DataFrame、合并、预处理、分组和案例分析
第 6 节	OpenCV 图像处理	图像基础、滤波、边缘检测、图像变换和视觉案例

表 1: 《Python 科学计算》讲义整体结构

1 科学计算与 Python 编程基础

Python 科学计算的入门内容可以分成三部分：开发环境、代码组织和数据组织。开发环境决定代码在哪里运行；代码组织决定计算过程怎样写清楚；数据组织决定信息怎样保存和传递。后续的 NumPy、Pandas、Matplotlib 和 OpenCV 都是在这三件事的基础上继续展开。

1.1 课程定位：用 Python 组织科学计算流程

科学计算程序通常从一个具体任务开始：有一批数据，需要计算、分析、画图或处理图像。围绕这些任务，课程会陆续用到下面这些工具：

- ▶ 编程基础：理解 Python 的运行方式、代码组织方式和基本数据结构；
- ▶ NumPy：用数组表示向量、矩阵、样本和高维数据；
- ▶ SciPy：调用优化、插值、积分、线性代数、信号处理等科学计算工具；
- ▶ Matplotlib：把计算结果转换为曲线、散点、柱状图、三维图和动画；
- ▶ Pandas：对表格数据进行读取、清洗、合并、分组和统计；
- ▶ OpenCV：处理图像数据，完成滤波、边缘检测、图像变换等任务。

这些库先有一个大致印象即可。为了看清它们为什么会放在同一门课里，先把科学计算任务抽象成一个共同流程。

定义 1.1 (科学计算). 科学计算是把数学模型、实验数据或工程问题转化为数值算法，并借助计算机完成求解、模拟、分析和可视化的过程。在 Python 语境下，一个典型科学计算任务通常包含四步：

问题 → 数据表示 → 数值计算 → 结果解释。

变量、分支、循环和函数用来描述计算过程；列表、字典和集合用来组织小规模数据；矩阵、表格和图像规模变大以后，再分别交给 NumPy、Pandas、Matplotlib 和 OpenCV。写这些程序之前，先要有一个能稳定运行代码的环境。

1.2 开发环境：从解释器到 Notebook

常用开发环境各有分工。解释器适合立即试一行代码，Notebook 适合边计算边记录，脚本适合保存可以反复运行的程序，IDE 适合管理较大的工程。

工具	适合做什么	在科学计算中的位置
Python 解释器	快速运行一行或几行代码	验证语法、测试表达式、检查包是否安装
文本编辑器	编辑独立的 .py 文件	保存脚本、练习语法、编写小工具
Jupyter Notebook	代码、文字、公式、图形混排	做实验、记录分析过程、展示计算结果
Anaconda	管理 Python、库和环境	降低科学计算库安装难度
PyCharm 等 IDE	管理较大项目、调试、补全	组织多文件工程和长期项目

表 2: Python 科学计算常见开发环境

1.2.1 环境启动与检查

安装完成后，应先确认环境可用。最小检查可以分成四步：

1. 系统能找到 Python：在命令行检查版本；
2. 能安装和导入科学计算库：至少能导入 `numpy`；
3. 能启动交互式工具：能打开 Jupyter Notebook；
4. 能在编辑器或 IDE 中运行 .py 文件。

Listing 1: 环境可用性的最小检查

```

1 # 在命令行中检查
2 python --version
3
4 # 在 Python 或 Notebook 中检查
5 import sys
6 import numpy as np
7
8 print(sys.version)
9 print(np.__version__)

```

如果使用 Anaconda，常见做法是先安装 Anaconda，再从 Anaconda Navigator 或命令行启动 Notebook；如果使用 PyCharm，要确认项目解释器指向正确的 Python 环境。解释器选错时，常见现象是命令行能导入某个库，但 IDE 中提示找不到该库。

课程中的 Windows 演示还强调了一个朴素但重要的顺序：先安装 Python 并检查是否加入 PATH，再安装编辑器和 Notebook，最后安装科学计算库。命令行中如果能进入 Python 解释器、能执行脚本、能启动 Notebook，环境才算真正连通。

Listing 2: 命令行中的运行与安装检查

```

1 python --version
2 python hello.py
3 python -m pip install numpy scipy matplotlib jupyter
4 jupyter notebook

```

若下载第三方包很慢，可以把 `pip` 源临时或永久换到国内镜像。镜像配置解决的是“从哪里下载包”的问题，不改变 Python 代码本身。

Listing 3: pip 镜像配置的典型形式

```
1 [global]
2 index-url = https://mirrors.aliyun.com/pypi/simple/
3 [install]
4 trusted-host = mirrors.aliyun.com
```

1.2.2 Notebook 的基本 workflow

Notebook 把一次计算拆成若干单元。每个单元可以单独执行，适合记录探索过程。一个简单的工作流通常是：

1. 在单元格中写 Python 代码；
2. 执行单元格，观察输出；
3. 修改代码，再次执行；
4. 在代码之间穿插文字说明、公式和图形；
5. 最终形成可复现的实验记录。

Listing 4: Notebook 中常见的探索式代码

```
1 import math
2
3 radius = 2.0
4 area = math.pi * radius ** 2
5 area
```

Notebook 的一个重要特点是“状态会保留”。如果前面单元格定义了变量，后面的单元格可以继续使用它。这对探索很方便，但也容易造成顺序混乱。学习时应养成两个习惯：一是按从上到下的顺序组织内容；二是定期重启内核并从头运行，检查笔记是否真正可复现。

1.3 Python 如何组织代码

环境检查通过以后，才轮到代码本身。理解 Python 程序，不能只记零散语法，而要看一段程序由哪些层次组成。一个 Python 程序可以看成由五层结构组成：

表达式 $\subset$ 语句 $\subset$ 函数 $\subset$ 类/模块 $\subset$ 包/项目。

1.3.1 变量与基本类型

Python 使用变量名引用对象。`int`、`float`、`bool`、`str` 等基本类型是后续所有数据结构的基础。

定义 1.2 (对象与变量). 在 Python 中，变量不是固定类型的盒子，而是指向对象的名字。对象携带自己的类型和值。例如 `x = 3` 让名字 `x` 指向一个整数对象；再执行 `x = "python"` 时，`x` 改为指向字符串对象。

Listing 5: 基本类型示例

```

1  n = 10                # int
2  rate = 0.15           # float
3  ok = True             # bool
4  name = "numpy"        # str
5
6  print(type(n), type(rate), type(ok), type(name))

```

科学计算代码中常见的一类错误，是把“数值”和“文本形式的数值”混在一起。例如 42 是整数，"42" 是字符串；二者在屏幕上相似，参与运算时完全不同。"3.14" 也是字符串，不是浮点数；它能被打印和拼接，但不能直接参与数值运算。

布尔类型 bool 只有 True 和 False 两个常用值。Python 使用 and、or、not 表示与、或、非；如果有 C 语言背景，可以把它们分别对应到 &&、|| 和 ! 来理解。字符串 str 可以用单引号或双引号声明，只要前后匹配即可。

例 1.1. 如果从文件或网页中读取到 "42"，应先转换为整数或浮点数：

```

1  raw = "42"
2  value = int(raw)
3  print(value + 1)

```

文件读取如此，交互式输入也是如此。input 得到的结果总是字符串。用户看起来输入了数字，程序拿到的仍然是文本；要参与数值计算，必须先转成 int 或 float。

Listing 6: input 与类型转换

```

1  score_text = input("Please input score: ")
2  score = int(score_text)
3
4  if score >= 60:
5      print("pass")
6  else:
7      print("fail")

```

上面的例子把输入文本转成整数后，才能继续做成绩判断。类型转换解决“能不能计算”，分支结构解决“按什么规则处理计算结果”。

1.3.2 分支：让程序根据条件选择路径

分支结构用于把判断规则写进程序。Python 使用 if、elif、else 表达互斥或递进的判断路径。

Listing 7: 成绩分级的分支结构

```

1  score = 86
2
3  if score >= 90:
4      grade = "A"
5  elif score >= 80:

```

```
6     grade = "B"
7 elif score >= 60:
8     grade = "C"
9 else:
10    grade = "D"
11
12 print(grade)
```

这里的判断顺序不能随意交换。若先判断 `score >= 60`，那么 86 分会先落入及格分支，后面的 B 等级就没有机会执行。多分支结构通常要把更具体、更严格的条件放在前面。

视频流中的成绩例子还把分支封装成了一个等级函数。它比简单的及格判断多了一个中间等级，因此阈值顺序更不能写错。

Listing 8: 成绩等级函数

```
1 def score2grade(score):
2     if score >= 90:
3         grade = "Good"
4     elif score >= 75:
5         grade = "Mid"
6     elif score >= 60:
7         grade = "pass"
8     else:
9         grade = "not pass"
10    return grade
11
12 while True:
13     score = int(input("Please input score:"))
14     grade = score2grade(score)
15     print(grade)
```

这里的 `input`、`int`、`if/elif/else`、`return` 和 `while True` 共同组成一个完整的小程序：反复读入成绩，转成整数，调用函数得到等级，再打印结果。

在科学计算中，分支常用于处理边界情况：数据是否为空、参数是否合法、迭代是否收敛、文件是否存在、图像是否读取成功等。它们的共同点是：同一段程序在不同输入下不能走完全相同的路径。

如果同样的判断和计算要对一批数据反复执行，就需要循环。分支决定“走哪条路”，循环决定“这条规则重复多少次”。

1.3.3 循环：重复执行同一类操作

循环用于表达“对一批对象重复处理”或“在条件满足前持续计算”。Python 常用的循环结构包括 `for`、`while`，并可用 `break` 和 `continue` 控制循环流程。

Listing 9: for 循环与累加

```
1 values = [1, 4, 9, 16]
2 total = 0
3
```

```
4 for x in values:
5     total += x
6
7 print(total)
```

Listing 10: while 循环与停止条件

```
1 x = 1.0
2 eps = 1e-6
3 steps = 0
4
5 while abs(x * x - 2) > eps:
6     x = 0.5 * (x + 2 / x)
7     steps += 1
8
9 print(x, steps)
```

第二段代码换成了 **while**，因为循环次数事先并不知道。程序只知道停止条件：当 $x * x$ 与 2 的误差足够小时停止。它是牛顿迭代求 $\sqrt{2}$ 的简化形式，也代表很多数值算法的写法：给定初值，反复更新，直到误差满足要求。

break 和 **continue** 用来改变循环的正常推进方式。**break** 直接结束整个循环，**continue** 跳过本轮剩余语句并进入下一轮。它们常用于搜索、过滤和异常数据处理。

Listing 11: break 与 continue

```
1 values = [3, -1, 4, 0, 5]
2
3 for x in values:
4     if x < 0:
5         continue          # 跳过无效数据
6     if x == 0:
7         break              # 遇到停止标记
8     print(x)
```

判断、循环和更新规则组合在一起后，往往会形成一段可复用的计算过程。把这段过程命名出来，就得到函数。

1.3.4 函数：把可复用逻辑命名

如果一段逻辑会写第二遍，就应该考虑把它变成函数。函数名说明它做什么，参数说明它需要什么，返回值说明它算出了什么。这样以后改公式、改阈值、改数据来源时，不必在很多地方找同一段代码。

定义 1.3 (函数). 函数是带名字的计算过程。它接收输入参数，执行一组语句，并可以通过 **return** 返回结果。函数的价值在于减少重复、明确接口、隔离细节。

Listing 12: 把数值迭代封装为函数

```
1 def sqrt_newton(a, eps=1e-6, max_iter=100):
```

```

2     x = 1.0
3     for _ in range(max_iter):
4         if abs(x * x - a) <= eps:
5             return x
6         x = 0.5 * (x + a / x)
7     return x
8
9 print(sqrt_newton(2))

```

`a` 是待开方的数，`eps` 是误差阈值，`max_iter` 是最大迭代次数。函数设计时应尽量让输入和输出清晰；这样比把所有变量放在全局作用域中更容易调试和复用。

1.3.5 模块、包与导入

模块是 Python 组织代码的重要单位：一个 `.py` 文件就是一个模块，多个模块可以组成包。使用 `import`、`from ... import ...` 可以复用标准库、第三方库或自己编写的模块。

Listing 13: 三种常见导入方式

```

1 import math
2 from math import sqrt
3 import numpy as np
4
5 print(math.sin(math.pi / 2))
6 print(sqrt(9))

```

科学计算代码里有一些固定写法，例如 `import numpy as np`、`import pandas as pd`、`import matplotlib.pyplot as plt`。这些别名不是语法规定，但大家都这么写。照着习惯来，别人读你的代码会少猜很多。

自己写的文件也可以被导入。例如把成绩函数和类放在 `school.py` 中，另一个脚本就可以用下面三种方式复用：

Listing 14: 导入自定义模块的三种方式

```

1 import school as sc
2 from school import score2grade as s2g
3 from school import *
4
5 print(sc.score2grade(86))
6 print(s2g(86))

```

第一种保留模块命名空间，调用时要写 `sc.`；第二种只导入某个名字并起别名；第三种把模块中的公开名字直接放入当前命名空间，练习时能用，但工程代码中容易造成名字冲突。

1.3.6 类：把数据和行为放在一起

类适合描述“既有状态又有行为”的对象。对初学科学计算而言，类不是一开始的重点，但理解 `class`、`__init__`、属性和方法，有助于读懂库的 API。

Listing 15: 一个简单的学生类

```

1 class student:
2     def __init__(self, name="", weight=100):
3         self.name = name
4         self.weight = weight
5
6     def hello(self):
7         print("My name is " + self.name)
8
9 s1 = student("Tom", 150)
10 s1.hello()

```

注 1.1. NumPy 数组、Pandas 的 DataFrame、Matplotlib 的 Figure、OpenCV 读入的图像对象，背后都可以从“对象有属性、有方法”的角度理解。掌握类的基本语义，能降低后续阅读库文档的难度。

函数和类主要解决“计算过程怎样组织”。但程序并不只由过程构成，还要保存和传递数据。这里转向 Python 内置的数据结构。

1.4 Python 如何组织数据

会写语句以后，还要选择合适的数据结构。一个变量可以保存一个数；一串名字、一条记录、一批不重复标签，则应使用不同结构。下面先用 Python 内置结构处理小数据，再看它们怎样过渡到 NumPy 数组和 Pandas 表格。

1.4.1 序列：字符串、列表与元组

序列是一类“按位置排列”的数据结构，可以用下标访问。字符串、列表和元组都属于序列，但可变性不同。

结构	是否可变	典型用途
字符串 str	否	文本、路径、标签、列名
列表 list	是	保存一批同类或相关对象，适合增删改
元组 tuple	否	固定组合，例如坐标、返回多个结果

表 3: Python 序列结构

Listing 16: 序列访问与切片

```

1 names = ["NumPy", "SciPy", "Matplotlib", "Pandas", "OpenCV"]
2
3 print(names[0])
4 print(names[-1])
5 print(names[1:4])

```

切片 `start:stop:step` 是后续 NumPy 数组操作的基础。Python 中的切片通常“含左不含右”，即包含 `start`，不包含 `stop`。

列表是可变序列，适合做“还会继续增删”的数据容器。写练习时最常用的是增、删、改、查：

操作	含义
append(x)	在列表末尾追加一个元素
insert(i, x)	在指定位置插入一个元素
extend(xs)	把另一个序列中的元素逐个追加进来
pop(i)	删除并返回指定位置的元素，省略 <code>i</code> 时删除末尾元素
remove(x)	删除第一个值等于 <code>x</code> 的元素
sort()	原地排序；若不想修改原列表，可使用 <code>sorted</code>

表 4: 列表的常用方法

Listing 17: 列表的增删改查

```

1 tools = ["numpy", "scipy"]
2 tools.append("pandas")
3 tools.insert(1, "matplotlib")
4 tools[0] = "NumPy"
5 removed = tools.pop()
6
7 print(tools)
8 print(removed)
```

元组与列表相似，也按位置组织数据，但元组不可变。它适合表达固定组合，例如二维坐标、函数返回的多个结果和字典中可哈希的复合键。

Listing 18: 元组打包与拆包

```

1 point = (3, 4)
2 x, y = point
3
4 print(x, y)
```

序列适合按位置取数据。不过有些数据只靠位置不够清楚。例如一条课程记录中，0 号位置到底是课程名、章节号还是主题列表，需要额外约定。此时更适合用字段名来取值。

1.4.2 字典：用键组织信息

当数据不是简单的顺序表，而是“字段名到字段值”的对应关系时，字典更合适。字典可以直接表达一条记录、一个配置对象或一张查找表。

定义 1.4 (字典). 字典 `dict` 是键到值的映射。键必须可哈希，常见键包括字符串、数字、元组；值可以是任意对象。字典适合表达记录、配置、计数器和查找表。

Listing 19: 用字典表示一条课程记录

```
1 course = {  
2     "name": "Python Scientific Computing",  
3     "chapter": 1,  
4     "topics": ["environment", "syntax", "data structures"]  
5 }  
6  
7 print(course["name"])  
8 print(course["topics"][0])
```

在数据分析中，一行表格可以看成是一个字典：列名是键，单元格内容是值。Pandas 的 DataFrame 可以理解为“许多行记录”或“许多列序列”的组合。

字典的常用访问方式包括按键取值、用 `get` 提供默认值、遍历键值对，以及批量更新字段。

Listing 20: 字典常用操作

```
1 course["teacher"] = "unknown"  
2  
3 print(course.get("chapter", 0))  
4  
5 for key, value in course.items():  
6     print(key, value)  
7  
8 course.update({"chapter": 2, "tool": "NumPy"})
```

字典关心的是“这个字段对应什么值”。如果问题只关心某个元素是否出现、是否重复，就不需要字段和值的映射，集合更直接。

1.4.3 集合：关心成员而非顺序

集合 `set` 用于保存不重复元素。它不强调顺序，强调成员关系和集合运算。

Listing 21: 集合去重与交集

```
1 a = {"python", "numpy", "pandas", "python"}  
2 b = {"numpy", "scipy", "matplotlib"}  
3  
4 print(a)  
5 print(a & b)  
6 print(a | b)  
7 print(a - b)  
8 print(a ^ b)
```

集合在数据清洗中很常用。例如统计唯一标签、判断某个字段是否属于合法集合、比较两批样本是否有交集等。集合运算中，`&` 表示交集，`|` 表示并集，`-` 表示差集，`^` 表示对称差集。若要修改集合，可使用 `add`、`remove`、`discard` 等方法。

列表、字典、集合都可以单独使用。真实数据通常会把它们套在一起：一条记录用字典，多条记录用列表，记录中的某个字段又可能是一组标签。

1.4.4 嵌套结构：从树状草图到复杂数据

真实数据往往不是单层结构，而是嵌套结构。列表、字典、元组可以组合使用，用来表达树状结构、表格记录、配置文件和复杂对象。

例 1.2. 可以用“列表套字典”表示多条学生记录：

```
1 students = [  
2     {"name": "Alice", "math": 90, "python": 85},  
3     {"name": "Bob", "math": 78, "python": 92},  
4     {"name": "Carol", "math": 88, "python": 89},  
5 ]  
6  
7 for stu in students:  
8     avg = (stu["math"] + stu["python"]) / 2  
9     print(stu["name"], avg)
```

这种结构已经接近表格数据。等进入 Pandas 后，上述数据可以自然转换为 DataFrame，每个字典对应一行，每个键对应一列。

还有一类数据并不是简单地排成一行。目录有父子层级，网页之间互相链接，道路之间彼此连通。树适合表达父子层级，例如目录、课程章节和决策过程；图适合表达多对多连接，例如道路网络、社交关系和网页链接。Python 中可以先用嵌套字典或邻接表表达这些结构。

Listing 22: 用字典和列表表达一张小图

```
1 graph = {  
2     "A": ["B", "C"],  
3     "B": ["A", "D"],  
4     "C": ["A"],  
5     "D": ["B"]  
6 }  
7  
8 for node, neighbors in graph.items():  
9     print(node, neighbors)
```

1.4.5 递归、时间开销与数据结构选择

数据结构不是只影响代码写法，也影响运行时间。斐波那契数列是一个容易观察差异的例子。递归写法贴近数学定义，但会重复计算许多子问题；迭代写法只保留当前两项，时间开销更稳定。

Listing 23: 斐波那契数列的递归与迭代写法

```
1 def fib(n):  
2     if n <= 2:  
3         return 1  
4     return fib(n - 1) + fib(n - 2)  
5
```



```
6 def fib_iter(n):
7     prev, curr = 1, 1
8     k = 2
9     while k < n:
10         prev, curr = curr, prev + curr
11         k += 1
12     return curr
```

列表、集合和字典也有类似差异。列表按顺序保存元素，在头部插入元素需要移动后面的许多元素；集合和字典用哈希结构组织数据，成员判断和按键查找通常接近常数时间。课程中用 `time.clock()` 比较过列表头部插入、列表尾部追加和集合插入的耗时；新版 Python 中应使用 `time.perf_counter()` 做同类计时。

Listing 24: 用计时理解列表与集合的操作差异

```
1 import time
2
3 start = time.perf_counter()
4 L = []
5 for i in range(100000):
6     L.insert(0, i)
7 print(time.perf_counter() - start)
8
9 start = time.perf_counter()
10 S = set()
11 for i in range(100000):
12     S.add(i)
13 print(time.perf_counter() - start)
```

1.4.6 选择数据结构的原则

数据结构选择

- ▶ 如果数据按位置排列，并且需要保留顺序，优先考虑列表或元组；
- ▶ 如果数据是字段到值的对应关系，优先考虑字典；
- ▶ 如果只关心元素是否出现、是否重复，优先考虑集合；
- ▶ 如果数据是数值矩阵、向量或高维数组，后续应使用 NumPy 数组；
- ▶ 如果数据是带列名的二维表，后续应使用 Pandas DataFrame。

这也是本节与后续内容的连接点：Python 内置结构适合小规模、通用的数据组织；当进入大规模数值计算和表格分析时，需要 NumPy 和 Pandas 提供更高效、更专业的抽象。

1.5 复习与练习

练习 1.1. 用一句话解释 Python 解释器、Notebook 和 IDE 的区别，并各举一个适合使用它们的场景。

练习 1.2. 写一个函数 `mean(values)`，接收一个数字列表并返回平均值。要求当列表为空时返回 `None`。

练习 1.3. 用字典表示一本书的信息，至少包含书名、作者、年份和关键词列表。然后遍历输出所有关键词。

练习 1.4. 给定列表 `["numpy", "pandas", "numpy", "scipy"]`，使用集合得到其中不重复的库名，并判断 `"matplotlib"` 是否出现。

练习 1.5. 把三名学生的姓名和两门成绩组织为“列表套字典”的结构，并计算每名学生的平均分。思考：如果学生数量变成三万人，这种结构还是否方便？

2 NumPy 数组计算

一旦数据从几个数字变成一长列、一张表或一批样本，普通列表就不再合适。NumPy 把同类型数值放进形状明确的数组，并把计算一次性作用到整批数据上。本节讨论数组的创建、属性、索引、变形、拼接、运算、统计、筛选，以及两个小实验：可视化和 KNN 分类。

2.1 为什么科学计算需要 NumPy

Python 内置的列表可以保存一批数值，但列表本身不是为大规模数值计算设计的。列表中的元素可以是不同类型，运算通常需要写显式循环。例如，要把一个列表中的每个数都平方，初学者常写成：

Listing 25: 用 Python 列表逐个计算平方

```
1 values = [1, 2, 3, 4, 5]
2 squares = []
3
4 for x in values:
5     squares.append(x ** 2)
6
7 print(squares)
```

这段代码当然正确，但它的表达重点落在“怎样循环”上，而不是“要做什么计算”上。科学计算中，我们更希望直接表达数学操作：一批数整体平方、一批样本整体求和、矩阵按行求平均、满足条件的数据整体取出。NumPy 正是为这种数组化表达而设计的。

定义 2.1 (NumPy). NumPy 是 Python 科学计算的基础数组库。它提供多维数组对象 `ndarray`，以及围绕数组建立的创建、索引、变形、拼接、广播、线性代数、随机数和统计运算等功能。后续的 SciPy、Matplotlib、Pandas、OpenCV 都大量依赖 NumPy 数组作为数据基础。

Listing 26: 用 NumPy 对整批数据计算平方

```
1 import numpy as np
2
3 values = np.array([1, 2, 3, 4, 5])
4 squares = values ** 2
5
6 print(squares)
```

这段代码没有显式写循环，但每个元素都被平方了。NumPy 把循环放在底层数组运算内部完成，代码只保留数学表达式本身。

引入 NumPy 有两个理由。第一是速度：大量数值循环如果写在 Python 层，解释器需要逐个处理对象；NumPy 把同类型数据放进连续内存，并用底层实现完成批量运算。第二是存储：Python 列表保存的是对象引用，而 `ndarray` 保存的是统一类型的数据块，更适合保存大规模数值矩阵。

这两个理由在课程中分别称为“时间上优化”和“节省空间”。时间上，NumPy 的底层计算库主要用 C 语言实现，数组操作通常把循环交给底层完成；空间上，普通 Python 对象除了数值本身，还要保存对象类型等附加信息，而 NumPy 数组只保存统一类型的数据块和必要的数组元信息。因此同样是一批整数，`int8`、`int16`、`int32`、`int64` 会占用不同字节数，选择类型会直接影响内存。

要使用 NumPy，先要认识它承载数据的对象。这个对象不是普通列表，而是带有维度、形状和类型信息的数组。

2.2 ndarray: NumPy 的主要数组类型

`ndarray` 可以读作“n-dimensional array”，即 n 维数组。它可以表示一维向量、二维矩阵，也可以表示更高维的数据。例如，一张灰度图像可以看成二维数组，一批彩色图像可以看成四维数组。

定义 2.2 (ndarray). `ndarray` 是 NumPy 中保存同类型元素的多维数组。一个数组不仅包含数据本身，还包含形状 `shape`、维数 `ndim`、元素总数 `size`、数据类型 `dtype` 等元信息。

Listing 27: 从列表创建一维数组和二维数组

```
1 import numpy as np
2
3 a = np.array([1, 2, 3, 4])
4 b = np.array([
5     [1, 2, 3],
6     [4, 5, 6]
7 ])
8
9 print(a)
10 print(b)
```

一维数组常用来表示向量或一组观测值；二维数组常用来表示矩阵、表格中的纯数值部分，或样本矩阵。若把二维数组的行看成样本、列看成特征，则很多机器学习算法都可以写成数组运算。

注 2.1. NumPy 数组和 Python 列表有一个很大的区别：NumPy 数组通常要求元素类型一致。这个限制换来了更紧凑的内存布局和更快的批量运算。

2.2.1 数组的数据类型

数组中的元素类型由 `dtype` 表示。常见类型包括整数、浮点数、布尔值和字符串。创建数组时，NumPy 会根据输入自动推断类型，也可以通过 `dtype` 参数显式指定。

Listing 28: 查看和指定数组数据类型

```
1 a = np.array([1, 2, 3])
2 b = np.array([1, 2, 3], dtype=float)
```

```
3 c = np.array([True, False, True])
4
5 print(a.dtype)
6 print(b.dtype)
7 print(c.dtype)
```

当输入中同时出现整数和浮点数时，NumPy 通常会把结果提升为浮点类型；当数值和字符串混合时，可能会整体转换为字符串。这种自动转换虽然方便，但在科学计算中要特别留意，因为类型变化会影响后续运算。

例 2.1. 下面的数组看起来包含数字，但因为混入了字符串，整体会变成字符串数组：

```
1 x = np.array([1, 2, "3"])
2 print(x)
3 print(x.dtype)
```

如果后续希望做数值运算，应先保证输入数据是数值类型。

改变数组类型时，应使用 `astype`。不要把“修改 `dtype` 属性”当成类型转换：直接改 `array.dtype` 可能只是重新解释同一段底层字节，数据块本身并没有按新类型重新计算。真正需要把 `float64` 转成 `int32`、把字符串数字转成浮点数时，应生成一个转换后的新数组。

Listing 29: 用 `astype` 做真正的类型转换

```
1 x = np.array([1.2, 2.8, 3.5], dtype=np.float64)
2 y = x.astype(np.int32)
3
4 print(x.dtype, x.itemsize)
5 print(y.dtype, y.itemsize)
```

知道数组元素是什么类型，只是读数组的第一步。数组还能告诉我们它有几维、每个方向有多长、总共占多少内存。下面这些属性决定了后续能怎样索引、变形和计算。

2.3 数组属性：先看清数据的结构

拿到一个数组后，不应急着计算，而应先检查它的结构。NumPy 数组最常用的属性包括：

属性	含义	例子
ndim	数组维数	一维数组为 1，二维数组为 2
shape	每个维度的长度	形状 (2, 3) 表示 2 行 3 列
size	元素总数	形状为 (2, 3) 的数组有 6 个元素
dtype	元素数据类型	int32、float64 等
itemsize	单个元素占用字节数	float64 通常占 8 字节
nbytes	数组数据总字节数	通常等于 <code>size * itemsize</code>
strides	各维移动一步跨过的字节数	用来理解数组视图和内存步长
flags	数组内存布局信息	是否连续、是否拥有自己的数据等
base	视图所依附的原数组	若为 None，通常说明自己拥有数据

表 5: NumPy 数组常用属性

Listing 30: 查看数组的基本属性

```

1 x = np.array([
2     [1.0, 2.0, 3.0],
3     [4.0, 5.0, 6.0]
4 ])
5
6 print(x.ndim)
7 print(x.shape)
8 print(x.size)
9 print(x.dtype)
10 print(x.itemsize)
11 print(x.nbytes)
12 print(x.strides)
13 print(x.flags)
14 print(x.base)

```

注 2.2. 读数组时先看 `ndim`，再看 `shape`，最后看 `dtype`。维数、形状和类型清楚以后，索引、拼接、运算错误通常更容易定位。

`strides` 和 `base` 能解释为什么有些操作很快。比如切片和部分 `reshape` 操作可以只创建视图，让新数组共享原数组的数据块；这时改视图可能影响原数组。若必须隔离数据，应显式使用 `copy`。

这些属性不是孤立概念。实际写代码时，通常要把它们连着检查一遍，再做访问和转换。

2.4 实验上：数组结构检查与基础访问

数组实验先固定几个动作：创建数组，查看属性，做类型转换，再练索引和切片。后面很多错误都来自 `shape`、`dtype` 或下标没有看清。

Listing 31: 数组结构检查实验

```
1 import numpy as np
2
3 a = np.array([1, 2, 3, 4, 5])
4 b = np.array([
5     [1, 2, 3],
6     [4, 5, 6]
7 ], dtype=np.float64)
8
9 print(type(a), type(b))
10 print(a.dtype, b.dtype)
11 print(a.ndim, b.ndim)
12 print(a.shape, b.shape)
13 print(a.size, b.size)
14 print(b.itemsize, b.nbytes)
```

如果发现数组类型不是预期类型，可以使用 `astype` 做显式转换。转换时要注意：浮点数转整数会丢失小数部分，字符串转数值要求字符串本身必须能解释为数。

Listing 32: 数组类型转换

```
1 x = np.array([1.2, 2.8, 3.5])
2 y = x.astype(np.int32)
3
4 print(x)
5 print(y)
6 print(y.dtype)
```

基础访问要同时练一维和二维。一维数组用一个下标访问，二维数组用行列两个方向访问；下标写法应和数组形状一起看。

Listing 33: 基础访问实验

```
1 x = np.arange(12).reshape(3, 4)
2
3 print(x[0, 0])
4 print(x[-1, -1])
5 print(x[0])
6 print(x[:, 0])
7 print(x[1:, 1:3])
```

完成基础访问以后，还要会构造各种常见形状的数组。实际计算中，很少只靠手写列表输入所有数据。

2.5 数组生成：从手写数据到规则数据

实际计算中，数组不一定都来自手写列表。NumPy 提供了许多数组生成函数，用于快速构造零数组、一数组、等差序列、单位矩阵和随机样本。

2.5.1 由已有数据创建数组

最直接的方法是使用 `np.array` 把列表、元组或嵌套列表转换为数组。

Listing 34: 由嵌套列表创建二维数组

```
1 data = [  
2     [90, 86, 75],  
3     [88, 92, 79],  
4     [95, 90, 84]  
5 ]  
6  
7 scores = np.array(data)  
8 print(scores)  
9 print(scores.shape)
```

如果每一行长度不同，数组就不能自然形成规整的二维数值矩阵。科学计算中的数组应尽量保持矩形结构：二维数组的每一行列数一致，高维数组的每个方向长度明确。

2.5.2 零数组、一数组和指定值数组

`zeros`、`ones`、`full` 常用于初始化数组。

Listing 35: 创建特殊填充值数组

```
1 z = np.zeros((2, 3))  
2 o = np.ones((2, 3))  
3 f = np.full((2, 3), 7)  
4  
5 print(z)  
6 print(o)  
7 print(f)
```

这些数组常用于预分配结果、构造掩码、初始化参数。例如，在模拟迭代中，可以先创建一个全零数组保存每一步的误差，再逐步填入计算结果。

`empty` 会分配形状合适的空间，但不把内容初始化为 0；它适合马上被完整覆盖的结果数组。若希望新数组沿用已有数组的形状和类型，可以使用 `zeros_like`、`ones_like` 或 `empty_like`。

Listing 36: 根据已有数组创建同形状数组

```
1 x = np.arange(6).reshape(2, 3)  
2 z = np.zeros_like(x)  
3 o = np.ones_like(x)  
4 e = np.empty_like(x)  
5  
6 print(z)  
7 print(o)  
8 print(e.shape, e.dtype)
```

2.5.3 等差序列与线性空间

`arange` 与 Python 的 `range` 类似，但返回 NumPy 数组，并且可以使用浮点步长。`linspace` 则用于在闭区间上生成指定数量的等间隔点。

Listing 37: 使用 `arange` 和 `linspace` 生成序列

```
1 x1 = np.arange(0, 10, 2)
2 x2 = np.linspace(0, 1, 6)
3
4 print(x1)
5 print(x2)
```

`arange` 更强调步长, `linspace` 更强调点数。画函数图像时常用 `linspace`, 因为我们通常希望在一个区间内取固定数量的采样点。

Listing 38: 为函数图像生成采样点

```
1 x = np.linspace(-np.pi, np.pi, 200)
2 y = np.sin(x)
```

2.5.4 单位矩阵和随机数组

线性代数中常用 `eye` 创建单位矩阵; 实验和模拟中常用随机数生成数组。

Listing 39: 单位矩阵与随机数组

```
1 I = np.eye(4)
2 u = np.random.random((2, 3))
3 n = np.random.normal(loc=0, scale=1, size=(2, 3))
4 r = np.random.randint(0, 10, size=(2, 3))
5
6 print(I)
7 print(u)
8 print(n)
9 print(r)
```

注 2.3. 随机数实验最好设置随机种子, 例如 `np.random.seed(0)`。这样每次运行得到的随机结果一致, 便于复现实验、调试代码和比较不同算法。

数组创建出来以后, 通常不会保持原样。计算前常常要取子集、改形状、转置, 或者复制出一份避免误改原数据。

2.6 基本操作: 索引、切片、变形与转置

数组创建以后, 最常见的操作是取出一部分、改变形状、改变方向。NumPy 的这些操作与 Python 序列切片一脉相承, 但扩展到了多维数组。

2.6.1 一维数组的索引和切片

一维数组的索引与列表类似: 从 0 开始, 负数从末尾开始。切片仍采用“含左不含右”的规则。

Listing 40: 一维数组索引和切片

```
1 a = np.arange(10)
2
3 print(a[0])
4 print(a[-1])
5 print(a[2:7])
6 print(a[:2])
7 print(a[:-1])
```

2.6.2 二维数组的索引和切片

二维数组可以用两个索引分别表示行和列。形式 $x[i, j]$ 表示第 i 行第 j 列；形式 $x[i, :]$ 表示第 i 行；形式 $x[:, j]$ 表示第 j 列。

Listing 41: 二维数组索引和切片

```
1 x = np.array([
2     [1, 2, 3, 4],
3     [5, 6, 7, 8],
4     [9, 10, 11, 12]
5 ])
6
7 print(x[0, 0])
8 print(x[1, :])
9 print(x[:, 2])
10 print(x[:2, 1:3])
```

轴的直观理解

二维数组中，`axis=0` 沿着行的方向向下看，常得到“每一列”的结果；`axis=1` 沿着列的方向横向看，常得到“每一行”的结果。不要机械背诵“0 是行、1 是列”，而要结合结果形状理解。

2.6.3 变形: reshape

`reshape` 可以在元素总数不变的前提下改变数组形状。

Listing 42: 使用 reshape 改变数组形状

```
1 a = np.arange(12)
2 b = a.reshape(3, 4)
3 c = a.reshape(2, 2, 3)
4
5 print(b)
6 print(c.shape)
```

如果原数组有 12 个元素，可以变成 (3, 4)、(4, 3)、(2, 6)、(2, 2, 3) 等形状，但不能变成 (5, 3)，因为元素总数不匹配。

例 2.2. 用 -1 可以让 NumPy 自动推断某个维度：

```
1 a = np.arange(12)
2 b = a.reshape(3, -1)
3 print(b.shape)
```

这里第一个维度指定为 3，剩下的维度由 12 个元素自动推断为 4。

2.6.4 展平、转置和复制

`ravel` 和 `flatten` 都可以把多维数组变成一维数组；`T` 可以转置二维数组。

Listing 43: 数组展平和转置

```
1 x = np.arange(12).reshape(3, 4)
2
3 print(x.ravel())
4 print(x.flatten())
5 print(x.T)
```

需要注意的是，有些操作返回视图，有些操作返回副本。视图与原数组共享数据，修改视图可能影响原数组；副本拥有独立数据。初学时如果不确定是否会影响原数组，可以显式使用 `copy`。

Listing 44: 使用 `copy` 避免修改原数组

```
1 x = np.arange(5)
2 y = x.copy()
3 y[0] = 100
4
5 print(x)
6 print(y)
```

2.6.5 插入、追加和删除

课程中的基本操作还包括对数组结构做小规模调整，例如追加元素、插入元素和删除元素。NumPy 数组的大小本身是固定的，所以这类函数通常返回一个新数组，而不是在原数组上原地扩容。

Listing 45: 数组追加、插入和删除

```
1 a = np.array([1, 2, 3])
2
3 b = np.append(a, [4, 5])
4 c = np.insert(a, 1, 100)
5 d = np.delete(a, 0)
6
7 print(b)
8 print(c)
9 print(d)
```

二维数组也可以指定 `axis`。如果不指定 `axis`，许多函数会先把数组展平成一维再处理；如果指定 `axis=0`，通常表示按行处理；指定 `axis=1`，通常表示按列处理。

Listing 46: 二维数组按轴插入和删除

```
1 x = np.arange(6).reshape(2, 3)
2
3 row_added = np.insert(x, 1, [10, 20, 30], axis=0)
4 col_deleted = np.delete(x, 1, axis=1)
5
6 print(row_added)
7 print(col_deleted)
```

索引和变形处理的是单个数组。若数据分散在多个数组中，或者一个数组需要拆给不同步骤使用，就需要合并和分割。

2.7 数组合并与分割

合并与分割处理的是数组的组织方式：有时要把几块数据拼成一块，有时要把一块数据拆成几份。先看形状再写函数。只有拼接方向之外的维度能对齐，数组才拼得起来。

2.7.1 合并数组

`concatenate` 是最通用的拼接函数，`vstack` 和 `hstack` 是二维场景下常用的便捷写法。

Listing 47: 一维数组拼接

```
1 a = np.array([1, 2, 3])
2 b = np.array([4, 5, 6])
3
4 print(np.concatenate([a, b]))
```

Listing 48: 二维数组按行和按列拼接

```
1 x = np.array([[1, 2], [3, 4]])
2 y = np.array([[5, 6], [7, 8]])
3
4 print(np.concatenate([x, y], axis=0))
5 print(np.concatenate([x, y], axis=1))
6 print(np.vstack([x, y]))
7 print(np.hstack([x, y]))
```

对于二维数组，`axis=0` 拼接后行数增加，列数不变；`axis=1` 拼接后列数增加，行数不变。

`stack` 与 `concatenate` 的区别在于：`stack` 会新增加一个轴。若把两个形状为 $(2, 3)$ 的数组用 `stack` 合并，结果可以变成 $(2, 2, 3)$ 或 $(2, 3, 2)$ ，具体取决于新轴放在哪里。

Listing 49: 使用 `stack` 增加新轴

```
1 x = np.ones((2, 3))
```

```
2 y = np.zeros((2, 3))
3
4 s0 = np.stack([x, y], axis=0)
5 s2 = np.stack([x, y], axis=2)
6
7 print(s0.shape)
8 print(s2.shape)
```

2.7.2 分割数组

`split`、`vsplit`、`hsplit` 可以把数组拆成若干块。

Listing 50: 数组分割

```
1 x = np.arange(16).reshape(4, 4)
2
3 upper, lower = np.vsplit(x, [2])
4 left, right = np.hsplit(x, [2])
5
6 print(upper)
7 print(lower)
8 print(left)
9 print(right)
```

注 2.4. 合并和分割本质上都是沿某个轴重组数据。学习时建议每次操作后都打印 `shape`，用形状变化检查自己是否理解了操作方向。

前面的例子都由代码直接构造数组。真实任务中，数组更常见的来源是网页、文本文件或表格。外部数据进入 NumPy 之前，通常要先做字段选择和类型转换。

2.8 实验中：从网页或文件得到表格型数据

实际计算很少从手写小数组开始。更常见的是先从网页、文本文件或表格里拿到数据，再把需要计算的数值列整理成数组。进入 NumPy 以后，仍然要先检查数据长什么样，再决定如何计算。

从外部来源得到的数据往往先是字符串、列表或二维表格。进入 NumPy 计算前，应完成三件事：选出真正需要的字段，去掉缺失或非法记录，把字段转换为数值类型。

Listing 51: 把表格型文本数据转换为 NumPy 数组

```
1 rows = [
2     ["name", "area", "price"],
3     ["A", "87.5", "210"],
4     ["B", "63.0", "168"],
5     ["C", "101.2", "260"]
6 ]
7
8 numeric = []
```

```
9 for row in rows[1:]:
10     area = float(row[1])
11     price = float(row[2])
12     numeric.append([area, price])
13
14 data = np.array(numeric)
15 print(data)
16 print(data.dtype)
```

如果数据已经保存为纯数值文本，可以直接使用 `loadtxt` 或 `genfromtxt`。前者适合整齐、无缺失的数据；后者对缺失值和表头更宽容。

Listing 52: 从文本文件读取数值数组

```
1 # data.csv 内容示例:
2 # area,price
3 # 87.5,210
4 # 63.0,168
5 # 101.2,260
6
7 data = np.genfromtxt(
8     "data.csv",
9     delimiter=",",
10    skip_header=1
11 )
12
13 print(data.shape)
14 print(data.mean(axis=0))
```

注 2.5. 网页抓取、文件读取和数据清洗本身不属于 NumPy 的主要 API，但它们经常出现在数组计算之前。拿到真实数据后，第一步仍然是查看 `shape`、`dtype`，再决定如何计算。

同一个数组如果后面还要反复使用，就不要每次都从原始文本重新解析。NumPy 可以把数组保存成自己的二进制格式。二进制格式会保留形状和数据类型，读取速度也比文本快。

Listing 53: 保存和读取单个数组

```
1 data = np.array([
2     [87.5, 210],
3     [63.0, 168],
4     [101.2, 260]
5 ])
6
7 np.save("house_data.npy", data)
8 loaded = np.load("house_data.npy")
9
10 print(loaded.shape)
11 print(loaded.dtype)
```

如果一次要保存多个数组，可以使用 `np.savez`。读取后得到的对象像字典一样，用保存时给出的名字取数组。

Listing 54: 保存多个数组

```
1 X = data[:, 0]
2 y = data[:, 1]
3
4 np.savez("house_arrays.npz", area=X, price=y)
5
6 arrays = np.load("house_arrays.npz")
7 print(arrays["area"])
8 print(arrays["price"])
```

文本格式也可以保存数组，例如 `savetxt`。它的优点是人可以直接打开查看，缺点是类型信息和结构信息较弱，复杂数组不适合只靠文本保存。

Listing 55: 把数组保存成 CSV 文本

```
1 np.savetxt(
2     "house_data_out.csv",
3     data,
4     delimiter=",",
5     header="area,price",
6     comments=""
7 )
```

数组文件的选择

中间计算结果优先用 `.npy` 或 `.npz`，因为它们能保留形状和类型；需要给人查看、给表格软件打开时，再导出为 CSV。不要把“计算用格式”和“展示用格式”混在一起。

数据整理成数组后，才进入计算阶段。NumPy 的计算通常不写逐个元素的循环，而是直接对数组整体写表达式。

2.9 数组运算：向量化与广播

NumPy 好用，很大一部分原因在于向量化运算。写代码时面对的是整个数组，底层会把运算应用到每个元素或每组元素上。这样代码更短，也更接近数学公式。

2.9.1 数组与标量运算

数组和单个数字运算时，标量会作用到数组的每个元素。

Listing 56: 数组与标量运算

```
1 x = np.array([1, 2, 3, 4])
2
3 print(x + 10)
4 print(x * 2)
5 print(x ** 2)
```

```
6 print(1 / x)
```

这类操作常见于单位转换、归一化、缩放和平移。例如，把摄氏温度转换为华氏温度，就可以对整列温度一次性计算。

2.9.2 数组与数组运算

形状相同的数组之间可以逐元素运算。

Listing 57: 形状相同数组的逐元素运算

```
1 a = np.array([1, 2, 3])
2 b = np.array([10, 20, 30])
3
4 print(a + b)
5 print(a * b)
6 print(a / b)
```

这里的乘法是逐元素乘法，不是线性代数中的矩阵乘法。若要做矩阵乘法，应使用 `@` 或 `np.dot`。

Listing 58: 矩阵乘法

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[10, 20], [30, 40]])
3
4 print(A * B)
5 print(A @ B)
```

2.9.3 通用函数

NumPy 提供大量通用函数，通常称为 `ufunc`。它们可以直接作用在数组上，例如 `sin`、`cos`、`exp`、`log`、`sqrt`、`abs`。

Listing 59: 常见通用函数

```
1 x = np.linspace(0, np.pi, 5)
2
3 print(np.sin(x))
4 print(np.cos(x))
5 print(np.sqrt(np.arange(1, 6)))
6 print(np.exp(np.array([0, 1, 2])))
```

2.9.4 广播

广播允许形状不同但兼容的数组一起运算。最常见的例子是二维数组和一维数组相加：如果一维数组长度等于二维数组的列数，它可以被看成每一行都加上同一个向量。

定义 2.3 (广播). 广播是 NumPy 在运算时自动扩展较小数组形状的规则。它并不一定真的复制数据，而是在逻辑上让数组形状对齐，从而完成逐元素运算。

Listing 60: 二维数组与一维数组广播

```
1 X = np.array([
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9]
5 ])
6 v = np.array([10, 20, 30])
7
8 print(X + v)
```

广播非常强大，但也容易造成误解。判断广播是否合理时，可以从末尾维度开始对齐：相等的维度可以运算，某个维度为 1 时可以扩展，否则就不兼容。

例 2.3. 样本矩阵按列标准化可以写成：

```
1 X = np.array([
2     [1.0, 10.0],
3     [2.0, 20.0],
4     [3.0, 30.0]
5 ])
6
7 mean = X.mean(axis=0)
8 std = X.std(axis=0)
9 Z = (X - mean) / std
10
11 print(Z)
```

数组运算会得到另一组数组。有时这正是需要的结果；但在统计和分析中，更常见的是把一批数压缩成均值、极值、方差或其他指标。

2.10 聚合运算：把数组压缩为统计量

聚合运算把一批数据压缩成一个或多个统计量。例如总和、乘积、均值、中位数、最大值、最小值、方差、标准差和分位数都属于聚合。练习时可以按同一个顺序来：先对整个数组聚合，再指定 `axis` 按行或按列聚合。

Listing 61: 一维数组聚合

```
1 x = np.array([1, 3, 5, 7, 9])
2
3 print(np.sum(x))
4 print(np.prod(x))
5 print(np.mean(x))
6 print(np.median(x))
7 print(np.min(x))
8 print(np.max(x))
9 print(np.var(x))
10 print(np.std(x))
```

```
11 print(np.percentile(x, 50))
```

对二维数组做聚合，最容易混淆的是 `axis`。

Listing 62: 二维数组按轴聚合

```
1 scores = np.array([
2     [90, 86, 75],
3     [88, 92, 79],
4     [95, 90, 84]
5 ])
6
7 print(scores.mean())
8 print(scores.mean(axis=0))
9 print(scores.mean(axis=1))
```

若把每一行看成一个学生，每一列看成一门课程，则：

- ▶ `scores.mean()` 得到所有分数的总体平均；
- ▶ `scores.mean(axis=0)` 得到每门课程的平均分；
- ▶ `scores.mean(axis=1)` 得到每个学生的平均分。

聚合后的形状

聚合不是只看数值，还要看形状变化。对形状为 `(3, 4)` 的数组沿 `axis=0` 求和，结果形状通常是 `(4,)`；沿 `axis=1` 求和，结果形状通常是 `(3,)`。形状变化就是理解聚合方向的最好证据。

除了把数组压缩成一个统计量，NumPy 还提供累计聚合。累计求和 `cumsum` 和累计乘积 `cumprod` 会保留每一步的中间结果，适合处理前缀和、累计增长、路径长度等问题。

Listing 63: 累计聚合

```
1 x = np.array([1, 2, 3, 4])
2
3 print(np.cumsum(x))
4 print(np.cumprod(x))
```

聚合返回的是值。很多时候只知道最大值还不够，还要知道它出现在什么位置；排序和近邻问题也需要保留位置。这类问题使用 `arg` 类索引。

2.11 arg 索引运算：找位置而不只是找值

普通的 `max` 和 `min` 返回最大值和最小值；`argmax` 和 `argmin` 返回最大值和最小值所在的位置。很多算法不只需要知道“值是多少”，还需要知道“它来自哪里”。

Listing 64: `argmax` 和 `argmin`

```
1 x = np.array([3, 9, 1, 7])
2
3 print(np.argmax(x))
```

```
4 print(np.argmax(x))
5 print(np.min(x))
6 print(np.argmin(x))
```

二维数组也可以使用 `axis` 参数。

Listing 65: 二维数组中的 `arg` 索引

```
1 scores = np.array([
2     [90, 86, 75],
3     [88, 92, 79],
4     [95, 90, 84]
5 ])
6
7 print(np.argmax(scores))
8 print(np.argmax(scores, axis=0))
9 print(np.argmax(scores, axis=1))
```

不指定 `axis` 时, NumPy 会把数组展平后返回位置。若要把展平位置还原为二维坐标, 可以使用 `unravel_index`。

Listing 66: 把展平索引还原为多维坐标

```
1 idx = np.argmax(scores)
2 row, col = np.unravel_index(idx, scores.shape)
3
4 print(row, col)
5 print(scores[row, col])
```

`argsort` 返回排序后的索引, 这在最近邻、排名、筛选前若干个元素时非常常用。

Listing 67: 使用 `argsort` 得到排序位置

```
1 distance = np.array([2.4, 0.8, 3.1, 1.2])
2 order = np.argsort(distance)
3
4 print(order)
5 print(distance[order])
```

`sort` 返回排序后的值, `argsort` 返回排序后的索引。二者常一起理解: 如果只关心数值从小到大排列, 用 `sort`; 如果还要回到原始数组中取对应样本, 就必须保留 `argsort` 给出的下标。

Listing 68: `sort` 与 `argsort` 的区别

```
1 x = np.array([30, 10, 40, 20])
2
3 print(np.sort(x))
4 print(np.argsort(x))
5 print(x[np.argsort(x)])
```

当只需要前几个最小值或最大值时, 不一定要完全排序。`partition` 和 `argpartition` 可以把第 k 小的元素放到正确位置, 并把更小和更大的元素分到两侧, 常用于 Top- k 问题。

Listing 69: 用 `argpartition` 取前 `k` 个近邻

```
1 distance = np.array([2.4, 0.8, 3.1, 1.2, 0.5])
2 k = 3
3
4 nearest = np.argpartition(distance, k)[:k]
5 nearest_sorted = nearest[np.argsort(distance[nearest])]
6
7 print(nearest)
8 print(nearest_sorted)
9 print(distance[nearest_sorted])
```

`arg` 类函数根据数值返回位置。另一类常见任务是根据条件选择数据，例如取出大于均值的样本，或取出某一类标签对应的行。

2.12 比较、布尔索引与 Fancy Index

NumPy 数组可以整体做比较，比较结果是布尔数组。布尔数组可以进一步用于筛选数据。

2.12.1 数组比较

Listing 70: 数组比较生成布尔数组

```
1 x = np.array([1, 5, 2, 8, 3])
2
3 print(x > 3)
4 print(x == 2)
5 print(x != 5)
```

比较结果不是单个 `True` 或 `False`，而是与原数组形状相同的布尔数组。每个位置表示该元素是否满足条件。

2.12.2 布尔索引

把布尔数组放进索引位置，就可以取出满足条件的元素。

Listing 71: 使用布尔索引筛选数据

```
1 x = np.array([1, 5, 2, 8, 3])
2 mask = x > 3
3
4 print(x[mask])
5 print(x[x % 2 == 0])
```

多个条件组合时，不能直接使用 Python 的 `and`、`or`，而应使用 `&`、`|`，并为每个条件加括号。

Listing 72: 组合多个筛选条件

```
1 x = np.array([1, 5, 2, 8, 3, 10])
2
```

```
3 selected = x[(x > 3) & (x < 10)]
4 print(selected)
```

布尔数组不仅能用于筛选，还能用于计数和判断。课程中这一类操作经常和比较表达式连在一起使用。

Listing 73: 布尔数组的计数和整体判断

```
1 x = np.array([1, 5, 2, 8, 3, 10])
2 mask = x > 3
3
4 print(np.count_nonzero(mask))
5 print(np.any(mask))
6 print(np.all(mask))
```

`where` 可以返回满足条件的位置，也可以按条件从两个值中二选一。第一种用法适合找下标，第二种用法适合批量替换。

Listing 74: `where` 的两种常见用法

```
1 x = np.array([1, 5, 2, 8, 3, 10])
2
3 print(np.where(x > 3))
4 print(np.where(x > 3, x, 0))
```

2.12.3 Fancy Index

Fancy Index 指用整数数组或整数列表作为索引，一次取出多个指定位置的元素。

Listing 75: 使用整数数组进行 Fancy Index

```
1 x = np.array([10, 20, 30, 40, 50])
2 idx = np.array([0, 2, 4])
3
4 print(x[idx])
```

二维数组中，Fancy Index 可以按行取，也可以同时给出行索引和列索引。

Listing 76: 二维数组 Fancy Index

```
1 X = np.arange(12).reshape(3, 4)
2
3 print(X[[0, 2], :])
4 print(X[[0, 1, 2], [1, 2, 3]])
```

第二个表达式会取出 `X[0, 1]`、`X[1, 2]`、`X[2, 3]` 这三个位置的元素。

注 2.6. 布尔索引回答“哪些元素满足条件”，Fancy Index 回答“我要这些指定位置”。两者经常和 `argsort`、`argmax`、`where` 配合使用。

到这里，数组的生成、筛选、拼接和统计都已经具备。下面两个实验把这些操作放到更完整的任务中：先整理数据并画图，再用距离和投票做分类。

2.13 实验一：用 NumPy 支撑数据可视化

NumPy 本身不是绘图库，但可视化前通常需要先使用 NumPy 生成或整理数据。例如，画函数曲线时，用 `linspace` 生成横坐标，用通用函数生成纵坐标；画散点图时，用二维数组保存样本坐标。

Listing 77: 生成函数曲线数据

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-2 * np.pi, 2 * np.pi, 400)
5 y1 = np.sin(x)
6 y2 = np.cos(x)
7
8 plt.plot(x, y1, label="sin")
9 plt.plot(x, y2, label="cos")
10 plt.legend()
11 plt.show()
```

这里的分工很清楚：NumPy 准备数值数组，Matplotlib 把数组画出来。后面学 Matplotlib 时，可以先把大多数绘图函数理解成“接收数组并画图”。

实验中的分类可视化使用了 Iris 数据集。这个数据集通常通过 `sklearn` 的数据集接口读取，特征矩阵 `iris.data` 是二维数组，标签 `iris.target` 是一维数组。用 NumPy 的切片先选出两个特征，再按标签布尔筛选，就能画出不同类别的散点。

Listing 78: Iris 数据的 NumPy 切片与散点图

```
1 from sklearn.datasets import load_iris
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 iris = load_iris()
6 X = iris.data[:, :2]
7 y = iris.target
8
9 for label in np.unique(y):
10     points = X[y == label]
11     plt.scatter(points[:, 0], points[:, 1], label=iris.target_names[
12         label])
13
14 plt.legend()
15 plt.show()
```

再看一个散点数据的例子。假设有两类二维点，每个样本有两个特征，可以用形状为 `(n, 2)` 的数组保存坐标，用一维数组保存类别标签。

Listing 79: 生成二维散点样本

```
1 np.random.seed(0)
2
```

```

3 class0 = np.random.normal(loc=[0, 0], scale=0.5, size=(50, 2))
4 class1 = np.random.normal(loc=[2, 2], scale=0.5, size=(50, 2))
5
6 X = np.vstack([class0, class1])
7 y = np.array([0] * 50 + [1] * 50)
8
9 plt.scatter(X[y == 0, 0], X[y == 0, 1], label="class 0")
10 plt.scatter(X[y == 1, 0], X[y == 1, 1], label="class 1")
11 plt.legend()
12 plt.show()

```

这里同时用到了数组生成、拼接、布尔索引和切片。可视化不是孤立技能，它依赖前面所有数组操作。

更接近真实分类数据的情况是：数据已经是一张二维表，每行是一个样本，前几列是特征，最后一列是类别标签。此时难点不在画图命令本身，而在于先用 NumPy 把特征列和标签列拆出来，再按标签做布尔筛选。

Listing 80: 按标签拆分样本并绘制散点图

```

1 # data 的每一行格式为 [feature1, feature2, label]
2 data = np.array([
3     [5.1, 3.5, 0],
4     [4.9, 3.0, 0],
5     [6.2, 3.4, 1],
6     [5.9, 3.0, 1],
7     [6.9, 3.1, 2],
8     [6.7, 3.3, 2]
9 ])
10
11 X = data[:, :2]
12 y = data[:, 2].astype(int)
13
14 for label in np.unique(y):
15     points = X[y == label]
16     plt.scatter(points[:, 0], points[:, 1], label=f"class {label}")
17
18 plt.legend()
19 plt.show()

```

这段代码覆盖了实验中反复出现的几个动作：用切片取特征矩阵和标签向量，用 `astype` 保证标签是整数，用 `unique` 找类别，用布尔索引取每一类样本。

可视化把样本分布画出来，但它本身不做判断。若要让程序根据已有样本预测新样本类别，就需要把“距离最近的样本更相似”写成计算步骤。

2.14 实验二：用 NumPy 实现 KNN 的主要步骤

KNN，即 k 近邻算法，是理解数组计算的好例子。它的思想很直接：给定一个未知类别的样本，先计算它与训练集中所有样本的距离，找到距离最近的 k 个样本，再用这些邻居的多数

类别作为预测结果。

定义 2.4 (KNN). KNN 是一种基于距离的分类方法。训练阶段主要保存已有样本；预测阶段计算新样本与训练样本之间的距离，并根据最近的若干个样本投票决定类别。

假设训练数据 X_{train} 的形状是 (n, d) ，表示 n 个样本、每个样本 d 个特征；标签 y_{train} 的形状是 $(n,)$ 。一个待预测样本 x 的形状是 $(d,)$ 。

Listing 81: KNN 单个样本预测的 NumPy 实现

```
1 import numpy as np
2 from collections import Counter
3
4 def knn_predict_one(X_train, y_train, x, k=3):
5     diff = X_train - x
6     dist = np.sqrt(np.sum(diff ** 2, axis=1))
7     nearest = np.argpartition(dist, k)[:k]
8     votes = Counter(y_train[nearest])
9     return votes.most_common(1)[0][0]
```

这段代码不长，但把前面的数组操作串在了一起：

- ▶ $X_{\text{train}} - x$ 使用广播，把一个样本同时与所有训练样本相减；
- ▶ $\text{diff} ** 2$ 对每个差值平方；
- ▶ $\text{np.sum}(\dots, \text{axis}=1)$ 对每个训练样本的特征差平方求和；
- ▶ $\text{argpartition}(\text{dist}, k)[:k]$ 找到距离较小的 k 个样本位置，不必对全部距离完整排序；
- ▶ $y_{\text{train}}[\text{nearest}]$ 用 Fancy Index 取出邻居标签；
- ▶ `Counter` 和 `most_common` 完成多数投票。

例 2.4. 构造一个很小的二维分类数据集：

```
1 X_train = np.array([
2     [0.0, 0.0],
3     [0.2, 0.1],
4     [1.8, 2.0],
5     [2.0, 1.7]
6 ])
7 y_train = np.array([0, 0, 1, 1])
8
9 x = np.array([1.7, 1.8])
10 print(knn_predict_one(X_train, y_train, x, k=3))
```

这个样本离右上角的一类点更近，因此通常会被预测为类别 1。

实验最后把函数封装成类。类里保存 k 、训练样本和训练标签；`fit` 只负责记住训练数据，`predict` 遍历待预测样本，真正的单样本预测放在私有辅助方法中。这个结构接近许多机器学习库的接口。

Listing 82: 把 KNN 封装为类

```

1 class KNNClassifier:
2     def __init__(self, k=3):
3         self.k = k
4         self.X_train = None
5         self.y_train = None
6
7     def fit(self, X_train, y_train):
8         self.X_train = X_train
9         self.y_train = y_train
10
11    def predict(self, X_predict):
12        return np.array([self._predict(x) for x in X_predict])
13
14    def _predict(self, x):
15        distances = np.sqrt(np.sum((self.X_train - x) ** 2, axis=1))
16        votes = Counter(self.y_train[np.argsort(distances, self
17        .k)[:self.k]])
18        return votes.most_common(1)[0][0]

```

KNN 实验能把本节知识串起来：二维数组表示样本矩阵，一维数组表示标签；广播完成批量距离计算；聚合按行求每个样本的距离；`argsort` 给出近邻候选位置；Fancy Index 根据位置取标签；`Counter` 找到投票最多的类别。

一个完整的练习还应包含数据准备、训练测试划分和准确率检查。下面的代码只用 NumPy 写出主要计算过程，不依赖机器学习库。

Listing 83: KNN 的训练测试流程

```

1 def train_test_split_numpy(X, y, test_ratio=0.3, seed=0):
2     rng = np.random.default_rng(seed)
3     idx = rng.permutation(len(X))
4     test_size = int(len(X) * test_ratio)
5     test_idx = idx[:test_size]
6     train_idx = idx[test_size:]
7     return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
8
9 def knn_predict(X_train, y_train, X_test, k=3):
10    preds = []
11    for x in X_test:
12        preds.append(knn_predict_one(X_train, y_train, x, k=k))
13    return np.array(preds)
14
15 X_train, X_test, y_train, y_test = train_test_split_numpy(X, y)
16 y_pred = knn_predict(X_train, y_train, X_test, k=3)
17 accuracy = np.mean(y_pred == y_test)
18
19 print(y_pred)
20 print(y_test)

```

```
21 print(accuracy)
```

如果各特征量纲差异很大，KNN 前还应做标准化。否则距离会被数值范围较大的特征主导。

Listing 84: KNN 前的特征标准化

```
1 mean = X_train.mean(axis=0)
2 std = X_train.std(axis=0)
3
4 X_train_std = (X_train - mean) / std
5 X_test_std = (X_test - mean) / std
```

标准化时只能用训练集的均值和标准差，再把同样的变换应用到测试集。这样做能避免把测试集信息提前泄露到训练过程中。

2.15 复习与练习

练习 2.1. 创建一个包含 1 到 20 的一维数组，将它变形为 (4, 5) 的二维数组，并分别输出它的 `ndim`、`shape`、`size` 和 `dtype`。

练习 2.2. 使用 `linspace` 在区间 $[0, 2\pi]$ 上生成 100 个点，计算这些点上的 $\sin x$ 和 $\cos x$ 。思考：为什么画函数图像时常用 `linspace` 而不是手写列表？

练习 2.3. 给定一个形状为 (5, 3) 的成绩数组，分别计算每门课的平均分和每个学生的平均分，并解释两个 `axis` 参数的含义。

练习 2.4. 生成一个包含 10 个随机整数的一维数组，找出其中最大值、最大值位置、排序后的索引，以及大于平均值的所有元素。

练习 2.5. 构造两个形状为 (2, 3) 的数组，分别沿 `axis=0` 和 `axis=1` 拼接，观察结果形状如何变化。

练习 2.6. 阅读本节 KNN 代码，逐行写出每个中间变量的形状。重点解释为什么 `X_train - x` 能直接计算所有训练样本与待预测样本的差。

3 SciPy 科学计算

NumPy 解决的是数组表示和数组运算问题。进入 SciPy 以后，学习重点变成：已经有了数组数据以后，怎样调用现成的科学计算算法完成拟合、优化、插值、线性代数、积分、信号处理和傅里叶分析。SciPy 仍然以 NumPy 数组为主要输入输出，因此前一节中的 `shape`、`dtype`、切片、广播、聚合和绘图前的数据组织仍然要熟练使用。

SciPy 的学习顺序可以按“模块解决的问题”来理解。常数包和特殊函数包用于补足数学工具；优化模块用于求最小值、做最小二乘拟合和解方程；插值模块根据离散采样点构造函数近似；线性代数模块处理矩阵方程、行列式、特征值和解；积分模块处理定积分、多重积分和常微分方程；信号和傅里叶模块则把时间序列、图像和频域分析联系起来。

3.1 SciPy 的模块结构

SciPy 不是一个单独的大函数库，而是按问题类型组织成许多子模块。写程序时，通常只导入当前任务需要的子模块。

模块名	应用领域
<code>scipy.cluster</code>	聚类计算，例如 K-means
<code>scipy.constants</code>	常数和单位
<code>scipy.fftpack</code>	傅里叶变换
<code>scipy.integrate</code>	积分程序和常微分方程求解
<code>scipy.interpolate</code>	插值
<code>scipy.io</code>	数据输入输出
<code>scipy.linalg</code>	线性代数程序
<code>scipy.ndimage</code>	多维图像处理
<code>scipy.odr</code>	正交距离回归
<code>scipy.optimize</code>	优化、拟合和方程求解
<code>scipy.signal</code>	信号处理
<code>scipy.sparse</code>	稀疏矩阵
<code>scipy.spatial</code>	空间数据结构和算法
<code>scipy.special</code>	特殊数学函数
<code>scipy.stats</code>	统计

表 6: SciPy 常用子模块及其分工

Notebook 很适合练习 SciPy。可以为第三章建立 `ch3` 目录，并把不同主题分别放入若干 notebook 文件。每一组代码都应配合图形或输出检查结果，而不是只运行函数名。

3.2 常数包和特殊函数

科学计算中经常要用光速、普朗克常数、元电荷、圆周率、黄金比例等常量。直接手写这些数值容易出错，也无法说明单位和不确定度。SciPy 的 `constants` 模块把常用常量整理成可直接调用的对象。

Listing 85: 访问常用科学常量

```

1 from scipy import constants as C
2
3 print(C.c)           # vacuum speed of light
4 print(C.h)           # Planck constant
5 print(C.e)           # elementary charge
6 print(C.pi)          # pi
7 print(C.golden)      # golden ratio

```

如果只需要数值，直接访问属性即可；如果还要单位和误差，应使用物理常数字典 `physical_constants`。它的键是常量名，值通常是三元组：

(value, unit, uncertainty).

Listing 86: 带单位和不确定度的物理常量

```

1 from scipy import constants as C
2
3 print(C.physical_constants)
4 print(C.physical_constants["Boltzmann constant in Hz/K"])
5 print(C.find("Boltzmann"))

```

`find` 用于按关键词查找常量名。比如玻尔兹曼常数有不同单位版本，可以先按关键词搜索完整键名，再从物理常数字典中取值。这样比靠记忆拼写更可靠。

特殊函数包 `scipy.special` 提供普通 `math` 模块之外的数学函数，也包含更稳定的数值实现。一个典型例子是 $\ln(1+x)$ 。当 x 很小时，先算 $1+x$ 可能已经把微小增量舍掉。

Listing 87: `log1p` 与普通 `log` 的差别

```

1 import math
2 from scipy import special
3
4 print(math.log(1 + 1e-20))
5 print(special.log1p(1e-20))

```

`math.log(1+1e-20)` 可能得到 `0.0`，而 `special.log1p(1e-20)` 能保留 10^{-20} 量级的信息。这说明特殊函数的价值不仅在于函数种类更多，也在于数值稳定性更好。

Listing 88: 特殊函数中的取整、排列和组合

```

1 from scipy import special
2 import numpy as np
3
4 print(special.cbrt(9))
5 print(special.round(4.5))
6 print(special.round(4.456, 2))
7 print(special.perm(5, 3))
8 print(special.comb(5, 3))
9
10 print(int(3.5))

```

```

11 print(np.round(4.5))
12 print(np.ceil(4.1))
13 print(np.floor(4.9))

```

`special.perm(5, 3)` 计算排列数 A_5^3 , 结果为 60; `special.comb(5, 3)` 计算组合数 C_5^3 , 结果为 10。取整函数要特别小心: `int` 是截断小数部分, `round` 系列在边界值上有自己的规则, `ceil` 和 `floor` 分别向上、向下取整。写数值程序时, 边界值不能只靠直觉判断, 应实际运行并说明规则。

`help(special)` 可以查看特殊函数族。帮助信息中能看到 Bessel 函数、球 Bessel 函数、Hankel 函数及其导数等名称。初学阶段不必掌握每个函数的理论背景, 但要知道这类函数集中在 `scipy.special` 中。

3.3 优化: 最小二乘拟合、函数最小值和方程求解

优化问题的形式很多, 但共同目标是让某个函数取得尽可能小的值, 或者让某个误差函数尽可能接近零。在线性回归中, 给定一组点 (x_i, y_i) , 希望用直线

$$\hat{y} = ax + b$$

拟合数据。常用目标是让残差平方和最小:

$$J(a, b) = \sum_i (y_i - (ax_i + b))^2.$$

这就是最小二乘法。对于简单直线拟合, 参数可以直接用公式计算:

$$a = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad b = \bar{y} - a\bar{x}.$$

Listing 89: 手写最小二乘直线拟合

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-1, 1, 10)
5 y = 2 * x + np.random.normal(0, 0.3, x.shape)
6
7 a = np.sum((x - x.mean()) * (y - y.mean())) / np.sum((x - x.mean())
8               ** 2)
9
10 b = y.mean() - a * x.mean()
11
12 plt.scatter(x, y)
13 plt.plot(x, a * x + b)
14 plt.grid()
15 plt.show()

```

手写公式能帮助理解最小二乘的含义, 但实际问题中的模型可能不是直线, 也可能有多个参数。这时可使用 `scipy.optimize.leastsq`。它要求我们写出残差函数, 而不是直接写

平方和。

Listing 90: 使用 leastsq 做参数拟合

```
1 from scipy.optimize import leastsq
2
3 def residuals(p, x, y):
4     a, b = p
5     return y - (a * x + b)
6
7 p0 = [1, 0]
8 params, status = leastsq(residuals, p0, args=(x, y))
9 print(params, status)
```

优化模块还提供函数最小值和方程求解。若目标是让单变量函数取最小值，可以先定义函数，再调用 `fmin`。

Listing 91: 求函数局部最小值

```
1 import numpy as np
2 from scipy.optimize import fmin
3
4 def f(x):
5     return x ** 2 + 20 * np.sin(x)
6
7 result = fmin(f, 3)
8 print(result)
```

`fmin` 返回的是从初值附近搜索到的局部极小点。若目标函数有多个局部极小值，初值会影响结果。因此优化结果要配合函数图像检查。

Listing 92: 绘制目标函数观察极值位置

```
1 x = np.linspace(-10, 10, 200)
2 y = f(x)
3
4 plt.plot(x, y)
5 plt.grid()
6 plt.show()
```

求方程根时使用 `fsolve`。它寻找的是让函数等于零的点。

Listing 93: 使用 fsolve 求方程根

```
1 from scipy.optimize import fsolve
2
3 def g(x):
4     return x ** 2 - 2
5
6 root = fsolve(g, 1)
7 print(root)
```

优化算法背后有许多思想。梯度下降沿着负梯度方向移动，直观上就是“往下降最快的方向走”；牛顿法用一阶导数和二阶导数构造局部近似，迭代形式可写成

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

或在求根问题中写成

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

拟牛顿法和共轭梯度法进一步减少对 Hessian 矩阵的直接计算。课程中优化部分先强调这些算法的直觉，再用 SciPy 函数完成实际求解。

优化结果的检查

优化函数返回数值以后，不能立刻认为答案正确。至少要检查三件事：初值是否合理，目标函数图像或残差是否符合预期，返回信息中是否显示成功收敛。对于多峰函数，还要尝试多个初值。

3.4 实验：最小二乘法和梯度下降

最小二乘实验把公式、代码和图形放在一起。第一步生成带噪声的散点，第二步手工计算拟合直线，第三步用 SciPy 的 `leastsq` 做同样的拟合，最后把散点和拟合曲线画在一张图上。

Listing 94: 最小二乘实验的基本流程

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import leastsq
4
5 x = np.arange(0, 10)
6 y = 3 * x + np.random.normal(0, 3, x.shape)
7
8 def residuals(p, x, y):
9     k, b = p
10    return y - (k * x + b)
11
12 p, _ = leastsq(residuals, [1, 0], args=(x, y))
13 k, b = p
14
15 plt.scatter(x, y)
16 plt.plot(x, k * x + b)
17 plt.show()
```

梯度下降实验通常从一个简单的二次函数开始，例如

$$f(x) = (x - 3)^2.$$

程序从初值出发，反复按照导数方向更新：

$$x_{n+1} = x_n - \alpha f'(x_n),$$

其中 α 是步长。步长过小，收敛很慢；步长过大，可能震荡甚至发散。实验中可以把每一步的 x 保存到 `history`，再把这些点画在函数曲线上。

Listing 95: 手写一维梯度下降

```

1 def f(x):
2     return (x - 3) ** 2
3
4 def df(x):
5     return 2 * (x - 3)
6
7 x = -4.0
8 lr = 0.1
9 history = []
10
11 while True:
12     history.append(x)
13     x_new = x - lr * df(x)
14     if abs(x_new - x) < 1e-6:
15         break
16     x = x_new

```

这个实验的意义不在于替代 SciPy 的优化函数，而在于看懂迭代算法的停止条件、步长和路径。等到目标函数更复杂时，再交给 `scipy.optimize`。

3.5 插值计算

拟合通常带有误差模型，插值则强调通过已知采样点构造一个函数，使它能估计采样点之间的值。假设有若干离散点 (x_i, y_i) ，希望得到一个可调用函数 f ，从而对新的 x 计算 $f(x)$ 。

SciPy 中常用 `interp1d` 做一维插值。

Listing 96: 一维插值的基本用法

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import interpolate
4
5 x = np.linspace(0, 10, 11)
6 y = np.sin(x)
7
8 f = interpolate.interp1d(x, y, kind="linear")
9 xnew = np.linspace(0, 10, 100)
10 ynew = f(xnew)
11
12 plt.scatter(x, y)

```



```

13 plt.plot(xnew, ynew)
14 plt.show()

```

`kind` 控制插值方式。课程实验中比较了线性、零阶、最近邻、二次和三次插值。线性插值把相邻点用直线连起来；零阶和最近邻会形成阶梯状曲线；二次、三次插值曲线更平滑，但也更容易在数据稀疏或噪声较大时产生过冲。

Listing 97: 比较多种插值方式

```

1 for kind in ["linear", "zero", "nearest", "quadratic", "cubic"]:
2     f = interpolate.interp1d(x, y, kind=kind)
3     plt.plot(xnew, f(xnew), label=kind)
4
5 plt.scatter(x, y)
6 plt.legend()
7 plt.show()

```

注 3.1. `interp1d` 默认只在原始 x 的范围内插值。若给出的 `xnew` 超出原始区间，可能出现 `ValueError`。这不是程序坏了，而是插值和外推的边界不同。需要外推时，应明确设置外推策略。

3.6 线性代数

线性代数模块 `scipy.linalg` 与 NumPy 的线性代数功能类似，但 SciPy 中算法更完整，适合科学计算任务。常见操作包括解线性方程组、计算行列式、特征值特征向量、矩阵乘法和矩阵分解。

Listing 98: 线性方程组、行列式和特征值

```

1 import numpy as np
2 from scipy import linalg
3
4 A = np.array([[1, 1, 3],
5               [2, 5, 1],
6               [2, 3, 8]])
7 b = np.array([10, 8, 3])
8
9 x = linalg.solve(A, b)
10 print(x)
11 print(A.dot(x))
12 print(linalg.det(A))
13
14 vals, vecs = linalg.eig(A)
15 print(vals)
16 print(vecs)

```

`linalg.solve(A,b)` 求解 $Ax = b$ ，得到的 `x` 应再代回 `A.dot(x)` 检查是否接近 `b`。`linalg.det(A)` 计算行列式。若行列式接近零，方程组可能病态，数值解会对误差很敏感。

特征值分解返回两个对象：特征值数组和特征向量矩阵。对矩阵 A ，若 $Av = \lambda v$ ，则 λ 是特征值， v 是对应特征向量。实验中还会使用 `np.dot`、`np.diag` 和四舍五入检查分解结果。

Listing 99: 用特征分解重构矩阵的思路

```
1 vals, vecs = linalg.eig(A)
2 D = np.diag(vals)
3 A2 = vecs @ D @ linalg.inv(vecs)
4 print(np.round(A2, 3))
```

如果矩阵来自图像或二维数据，线性代数分解还可以用于降维、近似和压缩。课程实验中把图像读成数组，查看数组形状，并对二维通道或灰度图做矩阵处理。图像本质上也是数组，只是每个位置对应像素。

3.7 数值积分和常微分方程

数值积分回答的是“已知函数，怎样近似面积或体积”。最简单的想法是把曲线下方面积分成很多小条，再把小条面积相加。用 NumPy 可以通过离散采样和梯形公式近似积分。

Listing 100: 用采样点和梯形法近似积分

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def half_circle(x):
5     return np.sqrt(1 - x ** 2)
6
7 x = np.linspace(-1, 1, 1000)
8 y = half_circle(x)
9
10 area = np.trapz(y, x)
11 print(area)
12 plt.plot(x, y)
13 plt.show()
```

半圆面积应接近 $\pi/2$ 。若用 `scipy.integrate.quad`，可以直接对函数做自适应积分，并返回误差估计。

Listing 101: 使用 quad 做一维积分

```
1 from scipy import integrate
2
3 value, error = integrate.quad(half_circle, -1, 1)
4 print(value, error)
```

二重积分可以使用 `dblquad`。例如对半球函数

$$z = \sqrt{1 - x^2 - y^2}$$

在圆盘区域上积分，就能得到半球体积。代码中要特别注意内层积分上下限可以是函数。

Listing 102: 二重积分的函数边界

```

1 def half_sphere(y, x):
2     return np.sqrt(1 - x ** 2 - y ** 2)
3
4 value, error = integrate.dblquad(
5     half_sphere,
6     -1, 1,
7     lambda x: -np.sqrt(1 - x ** 2),
8     lambda x: np.sqrt(1 - x ** 2)
9 )
10 print(value, error)

```

积分模块还能求解常微分方程。课程实验中出现了三维轨道的数值积分：先写出状态变量随时间变化的方程，再用积分器产生一串轨迹点，最后用三维图显示轨道。

Listing 103: 常微分方程数值积分的基本结构

```

1 from scipy.integrate import odeint
2 import numpy as np
3
4 def system(state, t, a, b, c):
5     x, y, z = state
6     return [
7         a * (y - x),
8         x * (b - z) - y,
9         x * y - c * z
10    ]
11
12 t = np.arange(0, 30, 0.01)
13 state0 = [0.0, 1.0, 0.0]
14 track = odeint(system, state0, t, args=(10.0, 28.0, 3.0))

```

这类代码的关键是函数签名和变量顺序。若把 x, y, z 、参数或时间变量顺序写错，图形可能仍能画出，但已经不是原方程的解。

3.8 实验：图像数组与 SciPy 处理流程

第三章中间实验把矩阵、图像和 SciPy 函数联系起来。图像读入以后会成为数组：彩色图像通常是三维数组，形状类似 (height, width, channels)；灰度图是二维数组。对图像做处理，本质上就是对数组做变换。

Listing 104: 读入图像并检查数组属性

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 img = plt.imread("face.png")
5 print(type(img))
6 print(img.shape)

```

```
7 print(img.dtype)
8
9 plt.imshow(img)
10 plt.axis("off")
11 plt.show()
```

显示某一个通道或灰度化后的二维数组时，可以使用不同的颜色映射：

Listing 105: 显示灰度或单通道图像

```
1 gray = img.mean(axis=2)
2
3 plt.imshow(gray, cmap="gray")
4 plt.axis("off")
5 plt.show()
6 print(gray.shape)
```

图像处理实验不要求把所有算法都手写出来，而是让读者理解：图像数据可以进入 NumPy/SciPy 流程，矩阵分解、滤波、旋转、裁剪、通道选择都可以看成数组操作。后续 OpenCV 章节会更系统地处理图像，这里先建立“图像就是数组”的观念。

3.9 信号处理

信号处理从一维序列开始。一个信号可以看成随时间变化的数组。为了模拟真实数据，课程中先生成一个正弦信号，再加入随机噪声。

Listing 106: 生成带噪声的一维信号

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 t = np.arange(0, 20, 0.1)
5 x = np.sin(t)
6 noise = np.random.normal(0, 0.2, t.shape)
7 signal = x + noise
8
9 plt.plot(t, signal)
10 plt.show()
```

`scipy.signal` 提供滤波和卷积工具。中值滤波常用于压制突发噪声，Wiener 滤波会根据局部统计信息平滑信号。滤波前后应画在同一张图中比较，不能只看数组输出。

Listing 107: 中值滤波和 Wiener 滤波

```
1 from scipy import signal as sg
2
3 med = sg.medfilt(signal, 5)
4 wie = sg.wiener(signal)
5
6 plt.plot(t, signal, label="raw")
7 plt.plot(t, med, label="medfilt")
```

```
8 plt.plot(t, wie, label="wiener")
9 plt.legend()
10 plt.show()
```

信号处理也可以作用于图像。二维图像滤波时，窗口大小、边界处理和卷积核都会影响结果。课程中把一维信号和图像放在同一节，是为了说明二者背后的数据结构相同：都是数组，只是维度不同。

3.10 傅里叶变换

傅里叶变换用于把信号从时域转换到频域。时域图告诉我们信号随时间怎样变化，频域图告诉我们信号由哪些频率成分组成。若一个信号由多个正弦波叠加而成，在时域中可能看起来复杂，在频域中会出现对应的峰。

Listing 108: 构造多频率信号并做 FFT

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import fftpack
4
5 t = np.linspace(0, 5, 500)
6 x = np.sin(2 * np.pi * 1 * t) + 0.5 * np.sin(2 * np.pi * 2 * t)
7
8 X = fftpack.fft(x)
9 freq = fftpack.fftfreq(len(t), d=t[1] - t[0])
10
11 plt.plot(t, x)
12 plt.show()
13
14 plt.plot(freq, np.abs(X))
15 plt.show()
```

频域图中峰的位置对应信号中的主要频率。实际显示时，常只画正频率部分，因为实信号的频谱具有对称性。

Listing 109: 只观察正频率部分

```
1 mask = freq > 0
2 plt.plot(freq[mask], np.abs(X)[mask])
3 plt.show()
```

傅里叶变换还可以用于频域滤波。思路是先把信号变到频域，削弱不需要的频率成分，再用逆变换回到时域。

Listing 110: 频域滤波的基本结构

```
1 X_filter = X.copy()
2 X_filter[np.abs(freq) > 3] = 0
3
4 x_smooth = fftpack.ifft(X_filter).real
5
```

```

6 plt.plot(t, x, label="raw")
7 plt.plot(t, x_smooth, label="filtered")
8 plt.legend()
9 plt.show()

```

这段代码把高于某个阈值的频率成分置零。实际应用中阈值不能随便选，应结合采样率、信号含义和噪声来源判断。傅里叶变换不是为了画出漂亮频谱，而是为了在频域中看见时域中不容易分辨的结构。

3.11 实验：信号滤波和频域分析

最后一个实验把信号处理和傅里叶变换连起来。先构造由多个正弦波组成的信号，再叠加随机噪声；然后分别用时域滤波和频域滤波处理，并比较原信号、噪声信号和处理后的结果。

Listing 111: 信号处理综合实验

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
4 from scipy import fftpack
5
6 t = np.linspace(0, 5, 500)
7 clean = np.sin(2 * np.pi * t) + 0.5 * np.sin(2 * np.pi * 3 * t)
8 raw = clean + np.random.normal(0, 0.4, t.shape)
9
10 median = signal.medfilt(raw, 5)
11
12 F = fftpack.fft(raw)
13 freq = fftpack.fftfreq(len(t), d=t[1] - t[0])
14 F[np.abs(freq) > 5] = 0
15 freq_filtered = fftpack.ifft(F).real
16
17 plt.plot(t, raw, label="raw", alpha=0.5)
18 plt.plot(t, median, label="median")
19 plt.plot(t, freq_filtered, label="fft filter")
20 plt.legend()
21 plt.show()

```

这个实验串起了第三章后半部分的主线：信号是一维数组，滤波是对数组局部结构的处理，傅里叶变换把数组换到频率视角，逆变换再回到时域。若能解释每一行代码的输入形状和输出形状，就说明已经把 SciPy 与 NumPy 的衔接关系看清楚了。

3.12 统计分布与假设检验

科学计算中经常不只关心一个确定数值，还关心随机变量的分布、样本差异是否明显、两个变量是否相关。SciPy 的 `stats` 模块提供常见概率分布和统计检验。它与 NumPy 的关系很直接：NumPy 提供样本数组，`scipy.stats` 对数组计算概率、分位数、检验统计量和 p 值。

Listing 112: 正态分布的概率和分位数

```

1 from scipy import stats
2
3 normal = stats.norm(loc=0, scale=1)
4
5 print(normal.pdf(0))          # 密度函数
6 print(normal.cdf(1.96))      #  $P(X \leq 1.96)$ 
7 print(normal.ppf(0.975))     # 0.975 分位数

```

pdf 是概率密度, cdf 是累计概率, ppf 是累计分布函数的反函数。若 $\text{cdf}(1.96)$ 接近 0.975, 则 $\text{ppf}(0.975)$ 会回到接近 1.96 的位置。

离散分布也用同样思路。例如二项分布可以描述重复试验中成功次数的分布。

Listing 113: 二项分布的概率质量

```

1 binom = stats.binom(n=10, p=0.3)
2
3 for k in range(0, 11):
4     print(k, binom.pmf(k))
5
6 print(binom.cdf(3))          #  $P(X \leq 3)$ 

```

如果已经有样本数据, 可以先做描述性统计, 再决定是否做检验。

Listing 114: 样本的描述性统计

```

1 sample = np.array([8.1, 7.9, 8.4, 8.0, 8.2, 8.3, 7.8])
2
3 print(sample.mean())
4 print(sample.std(ddof=1))
5 print(stats.describe(sample))

```

ddof=1 表示计算样本标准差。它与默认的总体标准差不同, 分母是 $n - 1$ 。在做统计推断时, 样本标准差更常用。

单样本 t 检验用于判断样本均值是否与某个给定值有显著差异。下面的代码检验样本均值是否可以认为等于 8.0。

Listing 115: 单样本 t 检验

```

1 res = stats.ttest_1samp(sample, popmean=8.0)
2
3 print(res.statistic)
4 print(res.pvalue)

```

p 值不是“假设为真的概率”, 而是在原假设成立时, 观察到当前或更极端结果的概率。若 p 值很小, 说明当前样本在原假设下不常见, 因此倾向于拒绝原假设。阈值如 0.05 只是约定, 不能代替对数据来源和实验设计的判断。

两组独立样本可以使用独立样本 t 检验。若两组方差不一定相等, 通常把 equal_var 设为 False, 使用 Welch 检验。

Listing 116: 两独立样本 t 检验

```

1 group_a = np.array([8.1, 7.9, 8.4, 8.0, 8.2])
2 group_b = np.array([7.5, 7.7, 7.8, 7.6, 7.9])
3
4 res = stats.ttest_ind(group_a, group_b, equal_var=False)
5 print(res.statistic, res.pvalue)

```

变量之间的线性相关性可以用 Pearson 相关系数衡量：

Listing 117: Pearson 相关系数

```

1 x = np.array([1, 2, 3, 4, 5])
2 y = np.array([2.1, 4.0, 6.2, 7.9, 10.1])
3
4 r, p = stats.pearsonr(x, y)
5 print(r, p)

```

相关系数接近 1 表示强正线性关系，接近 -1 表示强负线性关系，接近 0 表示线性关系弱。相关不等于因果，尤其是在数据不是实验随机分组得到时，不能只凭相关系数解释因果机制。

统计检验的使用顺序

先画图 and 做描述性统计，再选择检验方法；先检查样本是否独立、变量类型和分布假设，再解释 p 值。统计函数给出的是计算结果，不自动保证问题问得正确。

3.13 复习与练习

练习 3.1. 导入 `scipy.constants as C`，输出 `C.c`、`C.h`、`C.e`、`C.pi`、`C.golden`，再用 `C.find("Boltzmann")` 查找玻尔兹曼相关常量。

练习 3.2. 比较 `math.log(1+1e-20)` 与 `special.log1p(1e-20)` 的输出，并解释浮点舍入为什么会影响前者。

练习 3.3. 生成一组带噪声的直线数据，分别用手写最小二乘公式和 `scipy.optimize.leastsq` 求拟合参数，并把两条拟合直线画在同一张图上。

练习 3.4. 定义函数 $f(x) = x^2 + 20 \sin x$ ，用 `fmin` 从不同初值出发求局部最小值。观察不同初值是否得到不同结果。

练习 3.5. 用 `interp1d` 比较 "linear"、"nearest"、"quadratic"、"cubic" 的插值曲线。尝试让 `xnew` 超出原区间，记录错误信息并解释原因。

练习 3.6. 构造一个 3×3 矩阵 A 和向量 b ，用 `linalg.solve(A,b)` 求解，并用 `A.dot(x)` 检查结果。

练习 3.7. 用 `np.trapz` 和 `integrate.quad` 计算单位半圆面积，比较结果与 $\pi/2$ 的误差。

练习 3.8. 构造一个带噪声正弦信号，分别使用 `signal.medfilt` 和 FFT 频域滤波处理。画出原始信号和处理后的信号，并说明两种方法的差别。

4 Matplotlib 绘图与可视化

前面几节已经能用 NumPy 表示数组，用 SciPy 完成拟合、插值、积分、信号处理等计算。计算结果如果只停留在数组或一串数字里，很难判断模型是否合理、误差是否异常、参数变化是否符合预期。Matplotlib 的任务，就是把这些数值对象转换成可观察的图形。

Matplotlib 最常用的入口是 `matplotlib.pyplot`。它把一张图、坐标轴、曲线、散点、标题和图例等对象包装成接近 MATLAB 风格的命令接口。学习时可以先把一幅图理解成三层：

数据数组 → 绘图函数 → 图形修饰与输出。

数据通常来自 NumPy 数组；绘图函数决定图形类型；标题、坐标轴、图例、网格和保存命令负责让图形能够被阅读和复用。

4.1 绘图环境与基本 workflow

在 Notebook 中使用 Matplotlib 时，通常先导入 NumPy 和 `pyplot`：

Listing 118: Matplotlib 的常用导入方式

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

`plt` 不是一个新的数学库，而是 Matplotlib 中绘图接口的常用别名。一个最小绘图过程只需要准备横坐标、纵坐标，再调用 `plot` 和 `show`。

Listing 119: 最小折线图

```
1 x = np.linspace(0, 20, 100)
2 y = np.sin(x)
3
4 plt.plot(x, y)
5 plt.show()
```

`np.linspace(0, 20, 100)` 在区间 $[0, 20]$ 上生成 100 个等距点，`np.sin(x)` 对数组逐元素求正弦。这里的 `x` 和 `y` 长度必须一致，因为每一个横坐标都要对应一个纵坐标。若只给 `plot(y)`，Matplotlib 会自动使用 `0, 1, 2, ...` 作为横坐标。

Notebook 会把图形直接显示在单元格输出区；在普通脚本中，`plt.show()` 会打开图形窗口。若要把结果写入文件，应在显示前调用 `savefig`：

Listing 120: 保存图形文件

```
1 plt.plot(x, y)
2 plt.savefig("sin.png", dpi=150)
3 plt.show()
```

`dpi` 控制输出图像分辨率。写报告或讲义时，保存图片比临时窗口更可靠，因为图片可以复用、比较和归档。

注 4.1. Matplotlib 官网的 gallery 很适合查例子。遇到不熟悉的图形时，不必从函数名开始死记，而应先在 gallery 中找相似图，再回到代码里理解数据形状和参数含义。

4.2 折线图：从曲线到可读图形

折线图适合表达一个变量随另一个变量连续变化的关系，例如时间序列、函数曲线和迭代误差。若需要在同一张图中比较多条曲线，可以连续调用多次 `plot`。

Listing 121: 在同一坐标轴上绘制多条曲线

```
1 x = np.linspace(0, 2 * np.pi, 100)
2 y1 = np.sin(x)
3 y2 = np.cos(x)
4
5 plt.plot(x, y1)
6 plt.plot(x, y2)
7 plt.show()
```

多条曲线放在一起以后，仅靠默认颜色往往不够。Matplotlib 允许在 `plot` 中设置颜色、线型、点标记、线宽和图例标签。

Listing 122: 线型、颜色、标记和图例

```
1 plt.plot(x, y1, "r-", linewidth=3, label="sin")
2 plt.plot(x, y2, "b--", marker="*", label="cos")
3 plt.legend()
4 plt.show()
```

"r-" 表示红色实线，"b--" 表示蓝色虚线。点标记可以使用 "o"、"*"、"+"、"x"、"s" 等。图例来自每条曲线的 `label` 参数，必须调用 `legend` 才会显示。

为了让读者知道图中每个量的含义，还需要补上标题、坐标轴标签、网格和显示范围。

Listing 123: 标题、坐标轴、范围和网格

```
1 plt.plot(x, y1, "r-", linewidth=2, label="voltage U1")
2 plt.plot(x, y2, "b-", linewidth=2, label="voltage U2")
3
4 plt.title("Voltage signal")
5 plt.xlabel("time / s")
6 plt.ylabel("voltage / V")
7 plt.xlim(0, 8)
8 plt.ylim(-1.1, 1.1)
9 plt.grid(True)
10 plt.legend()
11 plt.show()
```

`xlim` 和 `ylim` 不是改变数据，而是改变坐标轴显示区间。若曲线本身没有问题，但图像看起来被截断或过于拥挤，首先应检查坐标范围。

折线图的阅读顺序

先看横轴和纵轴表示什么，再看每条曲线的标签和样式，最后看局部变化。没有坐标轴标签的曲线只能说明形状，很难说明实际含义。

4.3 图形对象、子图和布局

简单图形可以直接使用 `plt.plot`，复杂图形则需要理解 `figure` 和坐标轴。一个 `figure` 是整张画布，里面可以放一个或多个坐标轴。设置画布大小时使用 `figsize`：

Listing 124: 设置画布大小

```
1 plt.figure(figsize=(8, 4))
2 plt.plot(x, y1, label="sin")
3 plt.plot(x, y2, label="cos")
4 plt.legend()
5 plt.show()
```

`figsize=(8,4)` 的单位是英寸。它影响整张图的比例，适合在报告、PPT 或论文中控制图形外观。

多个图形放在同一张画布中时，可以使用 `subplot`。三位数字 221 表示把画布分成 2×2 ，并选择第 1 个位置；后面的 222、223、224 分别选择其余位置。

Listing 125: 使用 `subplot` 创建四个子图

```
1 plt.subplot(221)
2 plt.plot(x, np.sin(x))
3
4 plt.subplot(222)
5 plt.plot(x, np.cos(x))
6
7 plt.subplot(223)
8 plt.plot(x, x ** 2)
9
10 plt.subplot(224)
11 plt.plot(x, np.sqrt(x))
12
13 plt.tight_layout()
14 plt.show()
```

子图的价值不只是节省空间，而是把可以比较的对象放在同一视觉上下文里。比如同一组 x 上的不同函数、同一信号的原始图和滤波图、同一数据集的不同特征组合，都适合用子图展示。

如果要更明确地控制坐标轴对象，可以使用 `subplots`：

Listing 126: 使用 `subplots` 获得坐标轴对象

```
1 fig, axes = plt.subplots(2, 2, figsize=(8, 6))
2
3 axes[0, 0].plot(x, np.sin(x))
4 axes[0, 1].plot(x, np.cos(x))
5 axes[1, 0].plot(x, x ** 2)
```

```
6 axes[1, 1].plot(x, np.sqrt(x))
7
8 plt.tight_layout()
9 plt.show()
```

这种写法把每个坐标轴作为对象保存下来。后续需要单独设置某个子图的标题、范围、图例或网格时，比反复切换 `plt.subplot` 更清楚。

4.4 散点图与分类数据

散点图适合观察二维样本之间的关系。它不把相邻点连成线，而是把每个样本作为一个点放到坐标平面上。对于机器学习中的样本特征、实验测量中的两列变量、图像像素通道之间的关系，散点图都很常用。

Listing 127: 基本散点图

```
1 from sklearn.datasets import load_iris
2
3 iris = load_iris()
4 X = iris.data
5 y = iris.target
6
7 plt.scatter(X[:, 0], X[:, 1])
8 plt.show()
```

`X[:, 0]` 和 `X[:, 1]` 分别取所有样本的第 0、第 1 个特征。散点图只能一次直接显示两个坐标维度，因此选择哪两个特征会影响图形可解释性。

如果样本带有类别标签，可以按类别分别绘制，使颜色自然对应不同类别。

Listing 128: 按类别绘制 Iris 散点图

```
1 plt.scatter(X[y == 0, 0], X[y == 0, 1], label="class 0")
2 plt.scatter(X[y == 1, 0], X[y == 1, 1], label="class 1")
3 plt.scatter(X[y == 2, 0], X[y == 2, 1], label="class 2")
4
5 plt.xlabel("feature 0")
6 plt.ylabel("feature 1")
7 plt.legend()
8 plt.show()
```

这里的 `y == 0` 会产生一个布尔数组，用它可以筛出某一类样本。这个写法与 NumPy 中的布尔索引完全一致。

`scatter` 还有几个常用参数：

- ▶ `s` 控制点的大小；
- ▶ `c` 或 `color` 控制颜色；
- ▶ `marker` 控制点的形状；
- ▶ `alpha` 控制透明度；

- `cmap` 控制数值映射到颜色时使用的颜色表。

Listing 129: 散点大小、颜色和透明度

```
1 plt.scatter(  
2     X[:, 0],  
3     X[:, 1],  
4     c=y,  
5     s=60,  
6     alpha=0.75,  
7     cmap="viridis"  
8 )  
9 plt.colorbar()  
10 plt.show()
```

当点很多且互相遮挡时，`alpha` 很有用。透明度不是装饰，它能让高密度区域和低密度区域在视觉上区分开来。

4.5 常用统计图形

Matplotlib 不只画曲线。课程中常见的图形包括柱状图、水平柱状图、分组柱状图、堆叠柱状图、直方图、图像显示、饼图、堆叠面积图和箱线图。选择图形时，应先看数据的语义：类别比较、分布观察、组成比例、二维矩阵或连续变化，对应的图形不同。

4.5.1 柱状图和水平柱状图

柱状图用于比较离散类别的数值。横轴通常是类别，纵轴是数量、成绩、频次或指标值。

Listing 130: 柱状图

```
1 cities = ["A", "B", "C", "D"]  
2 values = [35, 42, 28, 31]  
3  
4 plt.bar(cities, values)  
5 plt.xlabel("city")  
6 plt.ylabel("value")  
7 plt.title("Bar chart")  
8 plt.show()
```

当类别名称较长时，水平柱状图更容易阅读：

Listing 131: 水平柱状图

```
1 plt.barh(cities, values)  
2 plt.xlabel("value")  
3 plt.ylabel("city")  
4 plt.show()
```

若要比对两组或多组数据，可以使用分组柱状图。它的关键是把每组柱子的横坐标错开一个宽度。

Listing 132: 分组柱状图

```

1 xpos = np.arange(len(cities))
2 width = 0.35
3 v1 = np.array([35, 42, 28, 31])
4 v2 = np.array([30, 38, 33, 27])
5
6 plt.bar(xpos - width / 2, v1, width=width, label="group 1")
7 plt.bar(xpos + width / 2, v2, width=width, label="group 2")
8 plt.xticks(xpos, cities)
9 plt.legend()
10 plt.show()

```

堆叠柱状图则强调总量由哪些部分组成，第二组柱子的底部应放在第一组柱子的顶部。

Listing 133: 堆叠柱状图

```

1 plt.bar(cities, v1, label="part 1")
2 plt.bar(cities, v2, bottom=v1, label="part 2")
3 plt.legend()
4 plt.show()

```

4.5.2 直方图和二维图像

直方图用于观察一组数值的分布。它先把数值区间分成若干箱，再统计每个箱中有多少样本。

Listing 134: 直方图

```

1 data = np.random.normal(0, 1, 1000)
2
3 plt.hist(data, bins=30)
4 plt.xlabel("value")
5 plt.ylabel("count")
6 plt.show()

```

bins 影响图形细节。箱数太少会掩盖分布形状，箱数太多会把随机波动放大。直方图适合看集中位置、离散程度、偏态和多峰结构。

二维数组可以用 **imshow** 显示成图像或热力图。颜色不再表示类别，而是表示数组元素的数值大小。

Listing 135: 使用 imshow 显示二维数组

```

1 Z = np.random.random((30, 40))
2
3 plt.imshow(Z, cmap="viridis")
4 plt.colorbar()
5 plt.show()

```

图像处理、相关矩阵、二维采样结果和热力分布都可以先组织成二维数组，再用 **imshow** 查看。若颜色条缺失，读者很难判断颜色对应的数值范围。

4.5.3 饼图、堆叠面积图和箱线图

饼图用于展示组成比例。它只适合类别数量较少、且确实强调“整体中的比例”的场景。

Listing 136: 饼图

```
1 shares = [30, 25, 20, 15, 10]
2 labels = ["A", "B", "C", "D", "E"]
3
4 plt.pie(shares, labels=labels, autopct="%1.1f%%")
5 plt.axis("equal")
6 plt.show()
```

`autopct` 用来显示百分比。格式字符串中百分号要写成 `%%`，否则会被当成格式控制符的一部分。

堆叠面积图强调多个部分随横坐标共同变化的趋势。

Listing 137: 堆叠面积图

```
1 x = np.arange(2016, 2021)
2 a = np.array([1, 3, 2, 4, 5])
3 b = np.array([2, 2, 3, 3, 4])
4 c = np.array([1, 1, 2, 2, 3])
5
6 plt.stackplot(x, a, b, c, labels=["A", "B", "C"])
7 plt.legend(loc="upper left")
8 plt.show()
```

箱线图用于比较多组数据的分布。它把中位数、四分位数和异常值放在同一个图形里，适合比较不同组的离散程度。

Listing 138: 箱线图

```
1 samples = [
2     np.random.normal(0, 1, 100),
3     np.random.normal(1, 1.5, 100),
4     np.random.normal(2, 0.8, 100)
5 ]
6
7 plt.boxplot(samples, labels=["A", "B", "C"])
8 plt.ylabel("value")
9 plt.show()
```

4.6 标注、图例与图像导出

一张图能画出来，不等于能用于报告或论文。可读图形至少要说明三件事：横轴是什么，纵轴是什么，图中关键位置在哪里。Matplotlib 的标注功能可以把重要点、阈值线和说明文字直接放到图上。

Listing 139: 给曲线添加标题、坐标轴和图例


```

1 x = np.linspace(0, 2 * np.pi, 200)
2 y1 = np.sin(x)
3 y2 = np.cos(x)
4
5 fig, ax = plt.subplots()
6 ax.plot(x, y1, label="sin(x)")
7 ax.plot(x, y2, label="cos(x)")
8 ax.set_title("Sine and cosine")
9 ax.set_xlabel("x")
10 ax.set_ylabel("value")
11 ax.legend()
12 plt.show()

```

如果图中有关键位置，应直接标出来。例如正弦函数在 $\pi/2$ 附近达到最大值：

Listing 140: 使用 annotate 标注关键点

```

1 x0 = np.pi / 2
2 y0 = np.sin(x0)
3
4 fig, ax = plt.subplots()
5 ax.plot(x, np.sin(x))
6 ax.scatter([x0], [y0], color="red")
7 ax.annotate(
8     "maximum",
9     xy=(x0, y0),
10    xytext=(x0 + 0.6, y0 - 0.3),
11    arrowprops={"arrowstyle": "->"}
12 )
13 ax.set_xlabel("x")
14 ax.set_ylabel("sin(x)")
15 plt.show()

```

阈值线常用于比较检测结果、统计边界和控制指标。水平线用 `axhline`，垂直线用 `axvline`。

Listing 141: 添加参考线

```

1 fig, ax = plt.subplots()
2 ax.plot(x, np.sin(x))
3 ax.axhline(0, color="gray", linewidth=1)
4 ax.axvline(np.pi, color="red", linestyle="--", label="pi")
5 ax.legend()
6 plt.show()

```

导出图片时，不要只依赖 Notebook 的显示结果。应显式指定文件名、分辨率和边界。报告中常用 PNG，论文和矢量排版中常用 PDF 或 SVG。

Listing 142: 保存高分辨率图像

```

1 fig, ax = plt.subplots()

```

```

2 ax.plot(x, np.sin(x), label="sin(x)")
3 ax.set_xlabel("x")
4 ax.set_ylabel("value")
5 ax.legend()
6
7 fig.savefig("sin_curve.png", dpi=300, bbox_inches="tight")
8 fig.savefig("sin_curve.pdf", bbox_inches="tight")

```

`dpi` 影响位图清晰度, `bbox_inches="tight"` 会尽量裁掉多余白边。保存前如果有中文、数学公式、图例和颜色条, 要打开导出的文件检查一次, 因为 Notebook 中能看清不代表导出的图片一定适合插入文档。

面向报告的图形检查

一张图导出前至少检查四点: 坐标轴是否有含义, 单位是否明确, 图例是否能区分类别, 关键结论是否能从图中直接读出。没有这些信息的图, 只能算中间调试结果。

4.7 动画可视化

静态图适合展示结果, 动画适合展示过程。排序、迭代优化、粒子运动、采样变化和实时数据更新, 都可以通过动画表达“状态如何一步一步改变”。

Matplotlib 的动画模块通常这样导入:

Listing 143: 动画模块导入

```

1 import matplotlib.animation as animation

```

一个简单动画由三部分组成: 初始化函数、更新函数和动画调度器。初始化函数负责创建第一帧; 更新函数接收帧编号并修改图形对象; `FuncAnimation` 负责反复调用更新函数。

Listing 144: 正弦曲线动画的基本结构

```

1 fig, ax = plt.subplots()
2 xdata, ydata = [], []
3 line, = ax.plot([], [], "ro")
4
5 def init():
6     ax.set_xlim(0, 2 * np.pi)
7     ax.set_ylim(-1, 1)
8     return line,
9
10 def update(frame):
11     xdata.append(frame)
12     ydata.append(np.sin(frame))
13     line.set_data(xdata, ydata)
14     return line,
15
16 frames = np.linspace(0, 2 * np.pi, 100)
17 ani = animation.FuncAnimation(
18     fig,

```

```

19     update,
20     frames=frames,
21     init_func=init,
22     interval=50,
23     blit=True
24 )
25 plt.show()

```

`interval` 是相邻两帧的时间间隔，单位是毫秒。`blit=True` 表示尽量只重绘变化部分，可以提高动画效率，但要求更新函数返回被修改的图形对象。

动画也可以用于散点图。课程中的泡泡图思路是：先随机生成每个点的位置、大小、增长速度和颜色；每一帧更新点的大小，并让某个点重新随机生成属性。

Listing 145: 泡泡散点动画的核心思路

```

1  n = 50
2  balls = {
3      "position": np.random.uniform(0, 1, (n, 2)),
4      "size": np.random.uniform(40, 70, n),
5      "growth": np.random.uniform(10, 20, n),
6      "color": np.random.uniform(0, 1, (n, 4))
7  }
8
9  fig, ax = plt.subplots()
10 sc = ax.scatter(
11     balls["position"][:, 0],
12     balls["position"][:, 1],
13     s=balls["size"],
14     c=balls["color"],
15     alpha=0.7
16 )
17
18 def update(number):
19     balls["size"] += balls["growth"]
20     index = number % n
21     balls["size"][index] = np.random.uniform(40, 70)
22     balls["position"][index] = np.random.uniform(0, 1, 2)
23     balls["color"][index] = np.random.uniform(0, 1, 4)
24
25     sc.set_sizes(balls["size"])
26     sc.set_offsets(balls["position"])
27     sc.set_color(balls["color"])
28     return sc,
29
30 ani = animation.FuncAnimation(fig, update, interval=20)
31 plt.show()

```

散点动画与曲线动画的差别在于更新方法不同。曲线常用 `set_data`，散点图常用 `set_offsets`、`set_sizes` 和 `set_color`。真正要抓住的是“图形对象已经创建，动画

函数只修改对象状态”。

若要保存动画，可以调用：

Listing 146: 保存动画

```
1 ani.save("test_animation.gif")
```

保存动画依赖本机可用的编码器或写入器。若保存失败，应先确认 Matplotlib 支持的 writer，而不是先怀疑绘图代码本身。

4.8 实验：排序过程的动画表示

排序动画实验把算法过程转成一串图形帧。数据中的每个元素画成一根柱子，柱子的高度表示数值，颜色表示当前算法正在比较、交换或已经确定的位置。

为了让动画函数只负责显示，实验通常先把算法运行过程保存成帧。每一帧是数据状态的一次快照。

Listing 147: 用对象保存柱子的值和颜色

```
1 class Data:
2     data_count = 30
3
4     def __init__(self, value):
5         self.value = value
6         self.color = "b"
7
8     def set_color(self, color):
9         self.color = color
```

选择排序的思想是：第 i 轮从未排序部分中找出最小值，放到位置 i 。为了把这个过程做成动画，每次比较时把参与比较的两根柱子改成不同颜色，并把这一状态复制到帧列表。

Listing 148: 选择排序动画帧的生成思路

```
1 import copy
2 import random
3
4 def selection_sort(data_set):
5     frames = [data_set]
6     ds = copy.deepcopy(data_set)
7
8     for i in range(0, Data.data_count - 1):
9         for j in range(i + 1, Data.data_count):
10            ds_r = copy.deepcopy(ds)
11            ds_r[i].set_color("r")
12            ds_r[j].set_color("k")
13            frames.append(ds_r)
14
15            if ds[j].value < ds[i].value:
16                ds[i], ds[j] = ds[j], ds[i]
```

```

17
18         frames.append(copy.deepcopy(ds))
19
20     return frames
21
22 data = list(range(1, Data.data_count + 1))
23 random.shuffle(data)
24 data_set = [Data(d) for d in data]
25 frames = selection_sort(data_set)

```

绘制某一帧时，只要取出所有元素的值和颜色，再调用 `bar`：

Listing 149: 把一帧排序状态画成柱状图

```

1 def plot_frame(data_set):
2     x = np.arange(Data.data_count)
3     height = [d.value for d in data_set]
4     colors = [d.color for d in data_set]
5
6     plt.cla()
7     plt.bar(x, height, color=colors)
8     plt.ylim(0, Data.data_count + 1)

```

最后用 `FuncAnimation` 把帧列表播放出来。

Listing 150: 播放排序动画

```

1 fig = plt.figure()
2
3 def animate(i):
4     plot_frame(frames[i])
5
6 ani = animation.FuncAnimation(
7     fig,
8     animate,
9     frames=len(frames),
10    interval=100
11 )
12 plt.show()

```

这个实验有两个值得注意的点。第一，算法本身和绘图函数分开写，方便调试；第二，保存帧时使用了深拷贝，否则后续交换会把已经保存的历史状态一起改掉。动画并不是给算法加装饰，而是把“算法运行中的状态序列”变成可观察对象。

4.9 图像像素与三维散点图案例

图像可以看成三维数组：高度、宽度和颜色通道。彩色图像常见形状为 `(height, width, 3)` 或 `(height, width, 4)`，其中三个颜色通道通常对应 R、G、B。Matplotlib 可以直接显示图像，也可以把像素值拿出来做二维或三维散点分析。

Listing 151: 读入并显示图像

```

1 import matplotlib.image as mpimg
2
3 img = mpimg.imread("test.jpg")
4 print(img.shape)
5
6 plt.imshow(img)
7 plt.axis("off")
8 plt.show()

```

`imshow` 显示的是图像本身；如果要研究像素通道之间的关系，可以把图像重塑成二维表，每一行表示一个像素。

Listing 152: 把图像整理成像素表

```

1 h, w, c = img.shape
2 X = img.reshape(h * w, c)
3
4 print(X.shape)
5 print(X[:5])

```

若 `X` 的前三列是 R、G、B 通道，可以先画两个通道的二维散点图：

Listing 153: 像素通道的二维散点图

```

1 plt.figure(figsize=(8, 6))
2 plt.scatter(X[:, 0], X[:, 1], alpha=0.1)
3 plt.xlabel("red")
4 plt.ylabel("green")
5 plt.show()

```

图中每一个点对应一个像素。若红色通道和绿色通道强相关，散点会沿某个方向聚集；若不同区域颜色差异明显，散点可能分成几个团块。对图像做聚类、颜色分割或降维前，这种图可以帮助判断颜色空间中的结构。

三维绘图需要引入 Matplotlib 的三维坐标轴。旧写法中常见 `Axes3D(fig)`，更推荐的写法是通过 `projection="3d"` 创建坐标轴。

Listing 154: RGB 像素的三维散点图

```

1 from mpl_toolkits.mplot3d import Axes3D
2
3 fig = plt.figure(figsize=(10, 10))
4 ax = plt.axes(projection="3d")
5 ax.scatter(X[:, 0], X[:, 1], X[:, 2], alpha=0.1)
6
7 ax.set_xlabel("red")
8 ax.set_ylabel("green")
9 ax.set_zlabel("blue")
10 plt.show()

```

三维散点图比二维图更接近 RGB 像素的完整表达，但点数很大时会变慢，也容易遮挡。实际分析时可以先裁剪图像或随机抽样：

Listing 155: 随机抽样后再绘制三维散点

```
1 idx = np.random.choice(len(X), size=5000, replace=False)
2 sample = X[idx]
3
4 fig = plt.figure(figsize=(8, 8))
5 ax = plt.axes(projection="3d")
6 ax.scatter(sample[:, 0], sample[:, 1], sample[:, 2], alpha=0.2)
7 plt.show()
```

`alpha` 在这里非常重要。像素点之间大量重叠，完全不透明会把密度差异盖住；透明度能让密集区域显得更深，稀疏区域显得更浅。

Matplotlib 与前面章节的衔接

Matplotlib 自己不负责产生科学计算结果。它接收 NumPy 数组、SciPy 计算输出、图像数组或实验过程中的状态序列，再把它们转换成图形。能否画出好图，首先取决于数据是否组织清楚。

4.10 复习与练习

练习 4.1. 生成 $x = \text{np.linspace}(0, 2 * \text{np.pi}, 100)$ ，分别绘制 $\sin x$ 和 $\cos x$ 。要求两条曲线使用不同颜色和线型，并添加标题、坐标轴标签、网格和图例。

练习 4.2. 用 `subplot` 或 `subplots` 在同一张画布中绘制 $\sin x$ 、 $\cos x$ 、 x^2 和 $\sqrt{x}$ 四个子图。比较两种写法在设置单个子图标题时的差别。

练习 4.3. 载入 Iris 数据集，任选两列特征绘制散点图。先画不区分类别的散点图，再用布尔索引按 `target` 分三类绘制，并添加图例。

练习 4.4. 构造两组城市指标数据，分别画分组柱状图和堆叠柱状图。解释两种图分别适合比较什么问题。

练习 4.5. 生成 1000 个正态分布随机数，使用不同 `bins` 参数绘制直方图。观察箱数变化如何影响分布形状。

练习 4.6. 写一个正弦曲线动画：每一帧向曲线中追加一个新点。要求使用 `FuncAnimation`，并说明 `frames`、`interval` 和 `init_func` 的作用。

练习 4.7. 实现一个简单的排序动画。可以选择选择排序或冒泡排序，把每次比较和交换保存为帧，并用柱状图显示当前状态。

练习 4.8. 读入一张彩色图片，查看其 `shape`，把像素整理成 $(n, 3)$ 的数组。分别绘制 R-G 二维散点图和 R-G-B 三维散点图，并尝试使用 `alpha` 或随机抽样改善显示效果。

5 Pandas 数据分析

NumPy 擅长处理同质数组，Matplotlib 擅长把数组画成图。真实数据分析还会遇到另一类问题：数据来自表格，列名有业务含义，行有索引，某些单元格缺失，某些列是文本、日期或类别。Pandas 正是为这类表格数据准备的工具。

Pandas 的常用导入方式是：

Listing 156: Pandas 的常用导入方式

```
1 import pandas as pd
2 import numpy as np
```

在科学计算流程中，Pandas 通常承担四件事：读取表格、整理字段、筛选和统计、把结果交给 Matplotlib 或 Seaborn 可视化。它不是替代 NumPy，而是在 NumPy 数组之上增加了索引、列名和缺失值处理能力。

5.1 从表格读取开始

Pandas 可以从多种来源读取数据。课程演示中使用过网页表格、剪贴板、CSV、Excel 等数据来源。最常见的入口函数包括：

- ▶ `pd.read_csv`：读取 CSV 文本文件；
- ▶ `pd.read_excel`：读取 Excel 表；
- ▶ `pd.read_clipboard`：读取剪贴板中的表格；
- ▶ `pd.read_html`：从网页中提取 HTML 表格；
- ▶ `pd.read_json`：读取 JSON 数据。

如果从网页复制了一张表，可以先用 `read_clipboard` 快速得到 DataFrame：

Listing 157: 从剪贴板读取表格

```
1 import pandas as pd
2
3 data = pd.read_clipboard()
4 data.head()
```

`head` 默认显示前 5 行，`tail` 默认显示后 5 行。刚读入数据时，不应急着统计，而要先查看行列、字段名、数据类型和缺失情况。

Listing 158: 读入数据后的基本检查

```
1 print(data.shape)
2 print(data.columns)
3 print(data.dtypes)
4
5 data.head()
6 data.tail()
7 data.info()
8 data.describe()
```

`shape` 告诉我们有多少行、多少列; `columns` 是列名; `dtypes` 是每列的数据类型; `info` 可以同时看到非空数量和类型; `describe` 给出数值列的基本统计量。表格分析中的许多错误, 都是在没有检查类型和缺失值时直接计算造成的。

注 5.1. Excel 中看起来像数字的列, 读入后可能是字符串; 百分号、货币符号、中文单位都会影响类型推断。遇到计算结果异常时, 先看 `dtypes`, 再考虑是否需要清洗。

5.2 Series: 带索引的一维数据

Pandas 的两个基本对象是 `Series` 和 `DataFrame`。`Series` 可以理解为带索引的一维数组。它由值和索引组成:

`Series` = 一列数据 + 一组索引.

Listing 159: 创建 Series

```
1 s = pd.Series([10, 20, 30, 40])
2 print(s)
```

如果没有指定索引, Pandas 使用 `0, 1, 2, ...` 作为默认索引。也可以显式指定索引:

Listing 160: 指定 Series 索引

```
1 s = pd.Series([35271, 38155, 23605], index=["BJ", "SH", "CQ"])
2 print(s)
3
4 print(s["BJ"])
5 print(s[0])
```

字典也可以直接转换为 `Series`。字典的键成为索引, 值成为数据:

Listing 161: 由字典创建 Series

```
1 gdp = {"BJ": 35271, "SH": 38155, "CQ": 23605}
2 s = pd.Series(gdp)
3 print(s)
```

`Series` 的一个重要特点是按索引对齐。两个 `Series` 做加法时, Pandas 不是简单按位置相加, 而是先对齐索引。

Listing 162: Series 运算中的索引对齐

```
1 s1 = pd.Series({"BJ": 35271, "SH": 38155, "CQ": 23605})
2 s2 = pd.Series({"BJ": 3016, "CQ": 2423, "GZ": 1530})
3
4 print(s1 + s2)
```

两个对象都有的索引会得到正常结果; 只在一边出现的索引会得到缺失值 `NaN`。这与 NumPy 数组的逐位置相加不同。索引对齐让表格数据更安全, 但也要求我们理解缺失值从哪里来。

5.3 DataFrame：二维表格对象

DataFrame 是带行索引和列索引的二维表格。可以把它看成多个 Series 按列组合在一起。

Listing 163: 由字典创建 DataFrame

```
1 df = pd.DataFrame({
2     "city": ["BJ", "SH", "CQ"],
3     "GDP": [35271, 38155, 23605],
4     "people": [3016, 2135, 2423]
5 })
6 print(df)
```

DataFrame 的列可以按列名访问：

Listing 164: 选择 DataFrame 的列

```
1 print(df["city"])
2 print(df[["city", "GDP"]])
```

一列返回 Series，多列返回 DataFrame。这个差别会影响后续方法调用和显示形式。
行选择常用 loc 和 iloc：

Listing 165: loc 与 iloc

```
1 df = df.set_index("city")
2
3 print(df.loc["BJ"])      # 按索引标签选择
4 print(df.iloc[0])        # 按整数位置选择
5 print(df.loc["BJ", "GDP"])
6 print(df.iloc[0, 0])
```

loc 面向标签，iloc 面向位置。若表格索引刚好是整数，二者容易混淆。学习 Pandas 时应养成明确写 loc 或 iloc 的习惯，不要依赖含糊的切片行为。

DataFrame 也可以重新设置索引、重排列、增加列和删除列。

Listing 166: 索引、列和新增字段

```
1 df = pd.DataFrame({
2     "city": ["BJ", "SH", "CQ"],
3     "GDP": [35271, 38155, 23605],
4     "people": [3016, 2135, 2423]
5 })
6
7 df = df.set_index("city")
8 df["GDP_per_person"] = df["GDP"] / df["people"]
9
10 print(df)
```

新增列通常来自已有列的计算。这里 `df["GDP"] / df["people"]` 是按行对齐后的向量化运算，不需要手写循环。

5.4 索引、切片与布尔筛选

表格分析离不开筛选。Pandas 中最常用的是布尔条件筛选：

Listing 167: 布尔筛选

```
1 df = pd.DataFrame({
2     "city": ["BJ", "SH", "CQ", "GZ"],
3     "GDP": [35271, 38155, 23605, 26927],
4     "people": [3016, 2135, 2423, 1530]
5 })
6
7 mask = df["GDP"] > 30000
8 print(df[mask])
```

多个条件组合时，要使用 `&`、`|`，并给每个条件加括号：

Listing 168: 多个条件筛选

```
1 result = df[(df["GDP"] > 30000) & (df["people"] > 2500)]
2 print(result)
```

这里不能写 `and`，因为每个条件返回的是一列布尔值，而不是单个布尔值。

排序常用 `sort_values` 和 `sort_index`：

Listing 169: 按值和索引排序

```
1 print(df.sort_values("GDP"))
2 print(df.sort_values("GDP", ascending=False))
3
4 df2 = df.set_index("city")
5 print(df2.sort_index())
```

`sort_values` 按某一列或多列排序，`sort_index` 按索引排序。统计前排序可以帮助检查极端值，展示结果时排序也能让表格更容易读。

5.5 缺失值、重复值与基础预处理

真实表格经常包含缺失值。Pandas 用 `NaN` 表示数值缺失，用 `isnull` 或 `notnull` 检查。

Listing 170: 缺失值检查

```
1 print(df.isnull())
2 print(df.isnull().sum())
```

处理缺失值常见两种方式：删除或填充。

Listing 171: 删除和填充缺失值

```
1 df_drop = df.dropna()
2 df_fill = df.fillna(0)
```

`dropna` 适合缺失较少且删除不会破坏分析的问题；`fillna` 适合用均值、中位数、固定值或业务规则补齐。不能机械地用 0 填所有缺失值，因为 0 本身可能有实际含义。

重复值可以用 `duplicated` 检查，用 `drop_duplicates` 删除。

Listing 172: 重复值处理

```
1 print(df.duplicated())
2 df_unique = df.drop_duplicates()
```

若只按部分列判断重复，可以传入 `subset`：

Listing 173: 按指定列判断重复

```
1 df_unique = df.drop_duplicates(subset=["city"])
```

预处理的目标不是让表格“看起来干净”，而是让后续统计的每一列都具有稳定含义：数值列确实是数值，类别列类别清楚，缺失和重复有明确处理规则。

5.6 apply: 把自定义规则作用到行或列

Pandas 的向量化运算能处理许多列计算，但有些字段需要自定义解析。课程中出现过形如

city:BJ,GDP:35271,PEOPLE:3016

的字符串字段。若要把它拆成多列，可以先写出单行的解析逻辑，再用 `apply` 应用到整列。

Listing 174: 用 apply 拆分字符串字段

```
1 df = pd.DataFrame({
2     "data": [
3         "city:BJ,GDP:35271,PEOPLE:3016",
4         "city:SH,GDP:38155,PEOPLE:2135",
5         "city:CQ,GDP:23605,PEOPLE:2423"
6     ]
7 })
8
9 result = df["data"].apply(
10     lambda line: pd.Series([
11         item.split(":")[1] for item in line.split(",")
12     ])
13 )
14 result.columns = ["city", "GDP", "people"]
15 print(result)
```

`line.split(",")` 先按逗号切成若干项，`item.split(":")[1]` 再取冒号后面的值。`pd.Series(...)` 让返回结果可以展开成多列。

`apply` 也可以作用于行。设 `axis=1` 时，函数接收的是每一行：

Listing 175: 按行计算新字段

```
1 df = pd.DataFrame({
2     "GDP": [35271, 38155, 23605],
3     "people": [3016, 2135, 2423]
4 })
5
6 df["ratio"] = df.apply(
```

```

7     lambda row: row["GDP"] / row["people"],
8     axis=1
9 )
10 print(df)

```

如果计算可以直接写成列运算，应优先使用列运算；`apply` 更适合复杂解析、条件规则或一行涉及多个字段的函数。

5.7 合并表格：merge、concat 和 combine

数据分析常常需要把多张表合并。合并方式不同，含义也不同。

`concat` 用于沿某个轴拼接对象。默认沿行方向追加：

Listing 176: 按行拼接 concat

```

1 df1 = pd.DataFrame({"city": ["BJ", "SH"], "GDP": [35271, 38155]})
2 df2 = pd.DataFrame({"city": ["CQ", "GZ"], "GDP": [23605, 26927]})
3
4 result = pd.concat([df1, df2], ignore_index=True)
5 print(result)

```

`ignore_index=True` 会重新生成连续索引。若保留原索引，拼接后可能出现重复索引。沿列方向拼接时使用 `axis=1`：

Listing 177: 按列拼接 concat

```

1 left = pd.DataFrame({"GDP": [35271, 38155]}, index=["BJ", "SH"])
2 right = pd.DataFrame({"people": [3016, 2135]}, index=["BJ", "SH"])
3
4 result = pd.concat([left, right], axis=1)
5 print(result)

```

按列拼接会根据索引对齐。若索引不一致，就会出现 NaN。可以通过 `join="inner"` 只保留共同索引。

Listing 178: concat 中的连接方式

```

1 result_outer = pd.concat([left, right], axis=1)
2 result_inner = pd.concat([left, right], axis=1, join="inner")

```

`merge` 更接近数据库中的表连接。它按照某个键列把两张表关联起来。

Listing 179: 使用 merge 按键合并

```

1 city = pd.DataFrame({
2     "city": ["BJ", "SH", "CQ"],
3     "GDP": [35271, 38155, 23605]
4 })
5 pop = pd.DataFrame({
6     "city": ["BJ", "SH", "GZ"],
7     "people": [3016, 2135, 1530]
8 })

```

```

9
10 print(pd.merge(city, pop, on="city", how="inner"))
11 print(pd.merge(city, pop, on="city", how="outer"))

```

`how="inner"` 只保留两张表都有的键;`how="outer"` 保留所有键,缺失处补 NaN;`left` 和 `right` 分别以左表或右表为准。

Listing 180: left/right 连接

```

1 print(pd.merge(city, pop, on="city", how="left"))
2 print(pd.merge(city, pop, on="city", how="right"))

```

如果两张表的键列名称不同,可以使用 `left_on` 和 `right_on`:

Listing 181: 键名不同的 merge

```

1 left = pd.DataFrame({"city_name": ["BJ", "SH"], "GDP": [35271,
2                        38155]})
3 right = pd.DataFrame({"city": ["BJ", "SH"], "people": [3016, 2135]})
4 result = pd.merge(left, right, left_on="city_name", right_on="city")
5 print(result)

```

`combine_first` 常用于“用一张表补另一张表的缺失值”。它按索引和列名对齐,优先保留调用者已有的非缺失值。

Listing 182: combine_first 补齐缺失值

```

1 a = pd.DataFrame({"GDP": [35271, np.nan]}, index=["BJ", "SH"])
2 b = pd.DataFrame({"GDP": [35000, 38155]}, index=["BJ", "SH"])
3
4 print(a.combine_first(b))

```

合并表格时要特别注意两件事:键是否唯一,合并后行数是否符合预期。多对多合并可能让行数突然膨胀,这通常不是 Pandas 出错,而是键关系没有检查清楚。

5.8 分箱、分组与聚合

分箱是把连续数值划分成离散区间。比如年龄、成绩、收入这类变量,可以先分成几个区间,再统计每个区间的人数或平均值。Pandas 中常用 `cut`:

Listing 183: 使用 cut 分箱

```

1 ages = pd.Series([12, 18, 23, 35, 46, 57, 68, 75])
2 bins = [0, 18, 35, 60, 100]
3
4 age_group = pd.cut(ages, bins)
5 print(age_group)
6 print(age_group.value_counts())

```

可以给区间指定标签:

Listing 184: 为分箱结果指定标签

```

1 labels = ["teen", "young", "middle", "old"]
2 age_group = pd.cut(ages, bins, labels=labels)
3 print(age_group)

```

`cut` 按给定边界分箱, `qcut` 按分位数分箱。前者适合业务区间明确的情况, 后者适合希望每个箱中样本数大致相近的情况。

分组合使用 `groupby`。先按某一列把数据分组, 再对每组计算统计量。

Listing 185: `groupby` 的基本用法

```

1 df = pd.DataFrame({
2     "city": ["BJ", "BJ", "SH", "SH", "CQ"],
3     "year": [2018, 2019, 2018, 2019, 2019],
4     "GDP": [33000, 35271, 36000, 38155, 23605]
5 })
6
7 print(df.groupby("city")["GDP"].mean())
8 print(df.groupby("city")["GDP"].sum())

```

多个分组键会产生多级索引:

Listing 186: 多键分组

```

1 result = df.groupby(["city", "year"])["GDP"].sum()
2 print(result)

```

若希望对不同列使用不同聚合函数, 可以使用 `agg`:

Listing 187: `agg` 聚合

```

1 result = df.groupby("city").agg({
2     "GDP": ["mean", "max", "min"],
3     "year": "count"
4 })
5 print(result)

```

日期字段也经常参与分组。读入后应先把字符串转换成日期类型:

Listing 188: 日期字段转换与按月统计

```

1 df["date"] = pd.to_datetime(df["date"])
2 df["month"] = df["date"].dt.month
3
4 monthly = df.groupby("month")["value"].sum()
5 print(monthly)

```

分组统计的结果通常可以直接交给 `Matplotlib` 或 `Seaborn` 画图。统计前, 先确认分组键的取值是否干净, 例如大小写、空格、缺失值和日期格式是否一致。

5.9 透视表与宽长表转换

分组聚合解决的是“按哪些键统计”。透视表解决的是“统计结果如何摆放”。同一份数据可以是长表，也可以是宽表。长表每一行是一条观测记录，适合筛选、分组和建模；宽表把某个分类变量展开成多列，适合对比、画矩阵图或交给需要矩阵输入的算法。

Listing 189: 长表形式的销售数据

```
1 sales = pd.DataFrame({
2     "city": ["BJ", "BJ", "BJ", "SH", "SH", "SH"],
3     "month": [1, 2, 3, 1, 2, 3],
4     "product": ["A", "A", "B", "A", "B", "B"],
5     "amount": [120, 150, 90, 200, 160, 180]
6 })
7
8 print(sales)
```

`pivot_table` 可以按行索引、列索引和值生成透视表。下面把城市放在行上，把月份放在列上，单元格中填销售额总和。

Listing 190: 用 `pivot table` 生成透视表

```
1 table = sales.pivot_table(
2     index="city",
3     columns="month",
4     values="amount",
5     aggfunc="sum",
6     fill_value=0
7 )
8
9 print(table)
```

如果同一组 `index` 和 `columns` 对应多条记录，必须指定聚合方式。这里使用 `sum`，表示同一城市同一月份的销售额相加；如果是价格、评分、温度，可能更应该使用 `mean`。

多个维度也可以同时放入透视表：

Listing 191: 多索引透视表

```
1 table = sales.pivot_table(
2     index=["city", "product"],
3     columns="month",
4     values="amount",
5     aggfunc="sum",
6     fill_value=0
7 )
8
9 print(table)
```

`pivot` 要求每个单元格只有一条记录；`pivot_table` 允许重复记录，并通过聚合函数合并。真实数据中常有重复键，所以更常用透视表函数。

宽表可以再变回长表。常用函数是 `melt`，它把多个列名压回一列，把对应数值压回另一列。

Listing 192: 用 melt 把宽表变成长表

```
1 wide = pd.DataFrame({
2     "city": ["BJ", "SH"],
3     "Jan": [120, 200],
4     "Feb": [150, 160],
5     "Mar": [90, 180]
6 })
7
8 long = wide.melt(
9     id_vars="city",
10    var_name="month",
11    value_name="amount"
12 )
13
14 print(long)
```

`id_vars` 是不参与展开的标识列，`var_name` 是原列名展开后所在的列，`value_name` 是原单元格数值所在的列。理解这三个参数，就能看清宽表和长表之间的转换关系。

交叉计数可以用 `crosstab`。它常用于类别变量之间的列联表，例如统计不同性别、不同舱位下的人数。

Listing 193: 类别变量交叉计数

```
1 df = pd.DataFrame({
2     "sex": ["F", "F", "M", "M", "F"],
3     "pclass": [1, 2, 2, 3, 1]
4 })
5
6 print(pd.crosstab(df["sex"], df["pclass"]))
```

长表和宽表的选择

需要筛选、分组、合并、建模时，长表通常更稳；需要横向比较多个类别、画热力图或构造矩阵时，宽表更直观。表格形状不是固定的，应该服务于下一步计算。

5.10 实验：Iris 数据预处理与相关性分析

Pandas 实验先用 Iris 数据集练习预处理。Iris 的特征包括花萼长度、花萼宽度、花瓣长度、花瓣宽度和类别。这个数据集适合练习表格读取、训练集划分、特征相关性和热力图。

Listing 194: 把 Iris 数据整理成 DataFrame

```
1 from sklearn.datasets import load_iris
2
3 iris = load_iris()
4 df = pd.DataFrame(iris.data, columns=iris.feature_names)
5 df["target"] = iris.target
6
7 df.head()
```

机器学习前常把特征和标签分开：

Listing 195: 分离特征和标签

```
1 X = df[iris.feature_names]
2 y = df["target"]
```

如果要划分训练集和测试集，可以使用 `train_test_split`：

Listing 196: 划分训练集和测试集

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X,
5     y,
6     test_size=0.2,
7     random_state=0
8 )
```

相关系数矩阵可以用 `corr` 计算，再用 Seaborn 画热力图：

Listing 197: 相关性热力图

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 corr = X_train.corr()
5 sns.heatmap(corr, cmap="RdYlBu", annot=True)
6 plt.show()
```

热力图中的颜色表示相关系数大小。接近 1 表示强正相关，接近 -1 表示强负相关，接近 0 表示线性相关弱。Iris 数据中，花瓣长度和花瓣宽度通常相关性较强，花萼宽度与其他特征的关系不同。相关性分析不是建模结论，但能帮助理解特征之间是否存在重复信息。

5.11 实验：泰坦尼克号数据分析

泰坦尼克号案例把 Pandas、Seaborn 和 Matplotlib 连在一起。数据中常见字段包括乘客编号、是否生还、舱位等级、姓名、性别、年龄、兄弟姐妹或配偶数量、父母子女数量、票号、票价、客舱和登船港口等。

读入数据后，第一步仍然是检查：

Listing 198: 泰坦尼克号数据的基本检查

```
1 df = pd.read_csv("titanic.csv")
2
3 df.head()
4 df.info()
5 df.describe()
6 df.isnull().sum()
```

Age、Cabin、Embarked 等字段经常含有缺失值。不同字段处理方式不同：年龄可以考虑用中位数或分组中位数填充；客舱缺失很多时，直接删除整列或转成“是否有客舱信息”可能更合适；登船港口缺失较少时，可以用众数填充。

Listing 199: 一种基础缺失值处理方式

```
1 df["Age"] = df["Age"].fillna(df["Age"].median())
2 df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
3 df["HasCabin"] = df["Cabin"].notnull()
```

查看生还率时，先用分组统计：

Listing 200: 按性别和舱位统计生还率

```
1 print(df.groupby("Sex")["Survived"].mean())
2 print(df.groupby("Pclass")["Survived"].mean())
```

再用图形表达：

Listing 201: 用 Seaborn 绘制生还率柱状图

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 sns.barplot(x="Sex", y="Survived", data=df)
5 plt.show()
6
7 sns.barplot(x="Pclass", y="Survived", data=df)
8 plt.show()
```

柱状图中的纵轴是平均生还值，因为 Survived 通常用 0 和 1 表示，平均值就是生还率。误差线表示估计的不确定性。

年龄分布可以用直方图观察：

Listing 202: 年龄分布直方图

```
1 df["Age"].hist(bins=30)
2 plt.xlabel("Age")
3 plt.ylabel("count")
4 plt.show()
```

如果要比不同类别下的年龄分布，可以按生还与否或性别分别绘制：

Listing 203: 按生还状态比较年龄分布

```
1 df[df["Survived"] == 0]["Age"].hist(alpha=0.5, bins=30, label="not
   survived")
2 df[df["Survived"] == 1]["Age"].hist(alpha=0.5, bins=30, label="
   survived")
3 plt.legend()
4 plt.show()
```

多个变量共同分析时，可以使用分组柱状图：

Listing 204: 性别和舱位共同影响

```
1 sns.barplot(x="Pclass", y="Survived", hue="Sex", data=df)
2 plt.show()
```

这个案例体现了 Pandas 的基本分析路线：读入数据，检查字段和缺失值，按业务问题选择分组键，计算统计量，再把统计结果可视化。图形中的差异要回到数据字段解释，不能只凭图形形状下结论。

Pandas 分析的基本顺序

先检查表格结构和类型，再处理缺失与重复；先用分组统计得到数值结果，再用图形展示。不要在不了解字段含义和缺失规则时直接画图。

5.12 复习与练习

练习 5.1. 从剪贴板、CSV 或 Excel 中读取一张表，分别输出 `shape`、`columns`、`dtypes`、`head`、`tail`、`info` 和 `describe`。

练习 5.2. 创建两个带城市索引的 `Series`，故意让索引不完全相同。做加法后观察 `NaN` 出现的位置，并解释索引对齐的含义。

练习 5.3. 创建一个包含城市、GDP、人口的 `DataFrame`。使用 `loc` 和 `iloc` 分别选择同一行同一列，比较标签索引和位置索引的区别。

练习 5.4. 对一个 `DataFrame` 做布尔筛选，筛出 GDP 大于某个阈值且人口大于某个阈值的行。尝试把 `&` 写成 `and`，观察错误信息。

练习 5.5. 构造含重复行和缺失值的表格，分别使用 `isnull`、`dropna`、`fillna`、`duplicated` 和 `drop_duplicates` 处理。

练习 5.6. 把形如 `city:BJ,GDP:35271,PEOPLE:3016` 的字符串列拆成三列。要求使用 `apply` 和 `pd.Series`。

练习 5.7. 构造两张含有城市键的表，分别用 `merge` 的 `inner`、`outer`、`left`、`right` 合并，比较结果行数和缺失值位置。

练习 5.8. 用 `pd.cut` 把年龄分成若干区间，再用 `value_counts` 统计每个区间的人数。随后用 `groupby` 计算每个年龄段的平均指标。

练习 5.9. 使用 Iris 数据集计算特征相关矩阵，并用 `sns.heatmap` 绘制热力图。说明哪两个特征相关性最强。

练习 5.10. 读取泰坦尼克号数据，分别统计按性别、舱位等级分组的生还率，并用柱状图展示。进一步画年龄分布直方图，观察年龄字段的分布特点。

6 OpenCV 图像处理与计算机视觉

OpenCV 是面向图像处理和计算机视觉的工具库。在 Python 科学计算环境中，它通常和 NumPy、Matplotlib 一起使用：OpenCV 负责读图、颜色变换、滤波、边缘检测、几何变换等底层图像操作；NumPy 负责把图像当作数组处理；Matplotlib 负责在 Notebook 中展示结果。

Listing 205: OpenCV 常用导入方式

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
```

在 OpenCV 中，一张普通图像本质上是一个 NumPy 数组。灰度图是二维数组，形状通常是 (height, width)；彩色图是三维数组，形状通常是 (height, width, channels)。常见图像的元素类型是 uint8，每个像素通道取值在 0 到 255 之间。

Listing 206: 读取图像并检查数组属性

```
1 img = cv2.imread("lena.jpg")
2
3 print(type(img))
4 print(img.shape)
5 print(img.dtype)
6 print(img.min(), img.max())
```

如果 cv2.imread 找不到文件，返回值不是报错，而是 None。因此在正式处理之前最好先检查：

Listing 207: 检查图像是否读入成功

```
1 img = cv2.imread("lena.jpg")
2 if img is None:
3     raise FileNotFoundError("image path is wrong")
```

6.1 图像显示、颜色通道与像素

OpenCV 默认按 BGR 顺序保存彩色图像，而 Matplotlib 默认按 RGB 顺序解释图像。因此，用 OpenCV 读入的彩色图如果直接交给 plt.imshow，颜色会不正常。正确做法是先转换通道顺序。

Listing 208: BGR 图像转成 RGB 后显示

```
1 img_bgr = cv2.imread("lena.jpg")
2 img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
3
4 plt.imshow(img_rgb)
5 plt.axis("off")
6 plt.show()
```

如果使用 OpenCV 自带窗口显示，颜色不需要转换：

Listing 209: OpenCV 窗口显示图像

```

1 img = cv2.imread("lena.jpg")
2 cv2.imshow("image", img)
3 cv2.waitKey(0)
4 cv2.destroyAllWindows()

```

`waitKey(0)` 表示一直等待按键；如果写成 `waitKey(30)`，表示等待约 30 毫秒，常用于视频逐帧显示。

图像数组可以像普通 NumPy 数组一样索引。对彩色图，`img[y, x]` 表示第 y 行、第 x 列的一个像素，返回三个通道值。注意图像坐标经常说 (x, y) ，而数组索引写作 `[y, x]`。

Listing 210: 读取像素和裁剪感兴趣区域

```

1 pixel = img_bgr[100, 200]
2 print(pixel)           # B, G, R
3
4 roi = img_bgr[80:240, 120:300]
5 plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
6 plt.axis("off")
7 plt.show()

```

彩色通道可以拆开，也可以重新合并：

Listing 211: 拆分和合并颜色通道

```

1 b, g, r = cv2.split(img_bgr)
2 img_bgr2 = cv2.merge([b, g, r])
3 img_rgb = cv2.merge([r, g, b])

```

在实际代码中，`cv2.split` 会复制数据，频繁使用时成本较高；如果只是为了取某个通道，也可以直接用数组切片，例如 `img[:, :, 0]`。

BGR 与 RGB

OpenCV 读入彩色图后，数组第三维的顺序是 B、G、R；Matplotlib 显示彩色图时按 R、G、B 解释。图像处理流程中最常见的颜色错误，就是忘记在这两个工具之间转换颜色顺序。

6.2 灰度图、直方图与阈值分割

灰度图只保留亮度信息，很多边缘检测、阈值分割和形状分析都先从灰度图开始。灰度图可以在读入时直接获得，也可以从彩色图转换。

Listing 212: 读入或转换灰度图

```

1 gray1 = cv2.imread("lena.jpg", cv2.IMREAD_GRAYSCALE)
2
3 img = cv2.imread("lena.jpg")
4 gray2 = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

```

灰度直方图统计每个灰度值出现的次数，可以帮助判断图像整体偏暗、偏亮，或者是否适合用某个阈值把目标和背景分开。

Listing 213: 灰度直方图

```

1 plt.hist(gray2.ravel(), bins=256, range=(0, 256))
2 plt.xlabel("gray value")
3 plt.ylabel("count")
4 plt.show()

```

最简单的分割方法是固定阈值。设定阈值 T 后，大于 T 的像素变成一个值，小于等于 T 的像素变成另一个值。

Listing 214: 固定阈值二值化

```

1 ret, binary = cv2.threshold(gray2, 127, 255, cv2.THRESH_BINARY)
2 print(ret)
3
4 plt.imshow(binary, cmap="gray")
5 plt.axis("off")
6 plt.show()

```

`cv2.threshold` 返回两个值：第一个是实际使用的阈值，第二个是二值化图像。对于固定阈值，返回的阈值就是给定的数；对于 Otsu 方法，返回的是算法自动选出的阈值。

Listing 215: Otsu 自动阈值

```

1 ret, binary = cv2.threshold(
2     gray2, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
3 )
4 print("otsu threshold:", ret)

```

如果前景是黑色、背景是白色，可以使用反向二值化：

Listing 216: 反向二值化

```

1 ret, binary_inv = cv2.threshold(
2     gray2, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
3 )

```

阈值分割的目标不是让图像“好看”，而是把后续算法需要的结构变清楚。例如手写数字识别中，最终需要的是数字笔画区域；车道线检测中，最终需要的是线状边缘。

6.3 轮廓、外接框与形态学

二值图得到以后，下一步经常不是直接识别，而是先找出图像中的连通区域。OpenCV 中的轮廓可以理解为目标边界上的点集合。手写数字、车牌字符、标志牌、证件区域等任务，常常需要先找轮廓，再根据面积、位置和外接框筛选目标。

Listing 217: 查找二值图中的外部轮廓

```

1 contours, hierarchy = cv2.findContours(
2     binary,
3     cv2.RETR_EXTERNAL,
4     cv2.CHAIN_APPROX_SIMPLE
5 )

```

```

6
7 print(len(contours))

```

RETR_EXTERNAL 表示只取最外层轮廓, 适合先找独立目标; CHAIN_APPROX_SIMPLE 会压缩轮廓点, 减少不必要的存储。若需要分析孔洞和层级关系, 可以使用其他轮廓检索模式。

轮廓找出来后, 可以计算面积和外接矩形。面积太小的轮廓通常是噪声; 外接框可以用于裁剪目标区域。

Listing 218: 按面积筛选轮廓并裁剪目标

```

1 boxes = []
2
3 for cnt in contours:
4     area = cv2.contourArea(cnt)
5     if area < 50:
6         continue
7
8     x, y, w, h = cv2.boundingRect(cnt)
9     boxes.append((x, y, w, h))
10
11 boxes = sorted(boxes, key=lambda item: item[0])
12
13 for x, y, w, h in boxes:
14     target = binary[y:y + h, x:x + w]
15     plt.imshow(target, cmap="gray")
16     plt.axis("off")
17     plt.show()

```

如果要把轮廓画回原图, 可以先复制原图, 再使用 `drawContours` 或 `rectangle`。

Listing 219: 绘制轮廓和外接框

```

1 vis = img.copy()
2
3 cv2.drawContours(vis, contours, -1, (0, 255, 0), 2)
4
5 for x, y, w, h in boxes:
6     cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 0, 255), 2)
7
8 plt.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))
9 plt.axis("off")
10 plt.show()

```

形态学操作常用来改善轮廓质量。腐蚀会让白色区域变小, 膨胀会让白色区域变大。开运算适合去小噪点, 闭运算适合连接断裂笔画或填小孔。

Listing 220: 腐蚀、膨胀、开运算和闭运算

```

1 kernel = np.ones((3, 3), np.uint8)
2
3 eroded = cv2.erode(binary, kernel, iterations=1)

```

```

4 dilated = cv2.dilate(binary, kernel, iterations=1)
5 opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
6 closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

```

轮廓检测对二值图质量很敏感。阈值过低会把背景噪声连成轮廓，阈值过高会把目标切断；形态学核太大又会把相邻目标粘在一起。因此轮廓任务通常要同时显示原图、二值图、形态学处理结果和最终轮廓图。

轮廓检测的检查顺序

先确认二值图中的前景和背景方向，再检查轮廓数量、面积分布和外接框位置。轮廓筛选不是只靠一行代码完成，而是根据目标尺寸、位置和形状逐步排除噪声。

6.4 实验：手写数字图像处理

手写数字图像通常有三个特点：背景比较简单，数字笔画和背景在灰度上有明显差异，目标区域可以通过阈值、裁剪、缩放等操作整理成统一格式。一个基本流程是：读入图像，转灰度，二值化，必要时反色，裁剪数字区域，缩放到模型要求的大小。

Listing 221: 单张手写数字的预处理

```

1 img = cv2.imread("digit.png")
2 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
3
4 _, binary = cv2.threshold(
5     gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
6 )
7
8 plt.imshow(binary, cmap="gray")
9 plt.axis("off")
10 plt.show()

```

如果数字周围有很多空白，需要找到非零区域，再裁剪出数字主体：

Listing 222: 根据非零像素裁剪数字主体

```

1 ys, xs = np.where(binary > 0)
2 y1, y2 = ys.min(), ys.max() + 1
3 x1, x2 = xs.min(), xs.max() + 1
4
5 digit = binary[y1:y2, x1:x2]
6 digit = cv2.resize(digit, (28, 28), interpolation=cv2.INTER_AREA)

```

如果图像背景里有网格、噪声或扫描痕迹，二值化后可能出现很多小点。可以使用形态学操作清理。开运算先腐蚀再膨胀，适合去掉小的白色噪点；闭运算先膨胀再腐蚀，适合填补笔画中的小孔。

Listing 223: 形态学清理二值图

```

1 kernel = np.ones((3, 3), np.uint8)
2 clean = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
3 clean = cv2.morphologyEx(clean, cv2.MORPH_CLOSE, kernel)

```

手写数字识别的输入不能直接是任意大小的图片。模型需要固定长度的特征向量，因此通常把处理后的数字图缩放到固定尺寸，然后展平成一维向量。

Listing 224: 把数字图像变成特征向量

```
1 digit = cv2.resize(clean, (28, 28), interpolation=cv2.INTER_AREA)
2 x = digit.reshape(1, -1).astype(np.float32) / 255.0
3 print(x.shape)           # (1, 784)
```

如果一批数字图片按类别放在文件夹中，可以循环读取，把每张图转成特征向量，同时记录标签。

Listing 225: 批量构造数字识别数据集

```
1 import os
2
3 data = []
4 labels = []
5
6 for label in range(10):
7     folder = f"digits/{label}"
8     for name in os.listdir(folder):
9         path = os.path.join(folder, name)
10        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
11        if img is None:
12            continue
13
14        _, binary = cv2.threshold(
15            img, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
16        )
17        binary = cv2.resize(binary, (28, 28), interpolation=cv2.
18            INTER_AREA)
19        data.append(binary.reshape(-1) / 255.0)
20        labels.append(label)
21
22 X = np.array(data, dtype=np.float32)
23 y = np.array(labels, dtype=np.int32)
24 print(X.shape, y.shape)
```

在这个实验中，图像处理的作用是把原始图片变成“机器学习模型能稳定读取的数据”。预处理不稳定时，后面的分类器即使写对了，也会因为输入特征不一致而效果很差。

6.5 实验：手写数字识别

一个完整的识别案例通常包含训练集、测试集、模型训练、预测和准确率评估。OpenCV 自带机器学习模块，也可以和 Scikit-learn 搭配。若使用 OpenCV 的 KNN，核心步骤如下。

Listing 226: OpenCV KNN 训练与预测

```
1 knn = cv2.ml.KNearest_create()
2 knn.train(X_train, cv2.ml.ROW_SAMPLE, y_train)
```

```

3
4 ret, results, neighbours, dist = knn.findNearest(X_test, k=3)
5 pred = results.ravel().astype(np.int32)
6 acc = (pred == y_test).mean()
7 print("accuracy:", acc)

```

ROW_SAMPLE 表示每一行是一条样本。训练特征矩阵的形状通常是 (n_samples, n_features); 标签是一维或列向量。手写数字图像如果统一成 28×28 , 展平后每个样本有 784 个特征。

识别单张新图片时, 必须使用和训练集一致的预处理函数。不能训练时反色、预测时不反色, 也不能训练时缩放到 28×28 , 预测时缩放到别的尺寸。

Listing 227: 用同一个函数处理训练图和测试图

```

1 def preprocess_digit(path):
2     img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
3     if img is None:
4         raise FileNotFoundError(path)
5
6     _, binary = cv2.threshold(
7         img, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
8     )
9     ys, xs = np.where(binary > 0)
10    if len(xs) == 0 or len(ys) == 0:
11        return np.zeros((1, 784), dtype=np.float32)
12
13    digit = binary[ys.min():ys.max() + 1, xs.min():xs.max() + 1]
14    digit = cv2.resize(digit, (28, 28), interpolation=cv2.INTER_AREA)
15    return (digit.reshape(1, -1).astype(np.float32) / 255.0)
16
17 x_new = preprocess_digit("new_digit.png")
18 ret, result, neighbours, dist = knn.findNearest(x_new, k=3)
19 print(int(result[0, 0]))

```

识别结果不好时, 先检查预处理图像, 而不是马上换模型。常见问题包括数字没有被完整裁剪、背景被当成前景、笔画太细、图像尺寸比例被拉坏、训练集和测试集的颜色方向相反。

手写数字案例的主线

手写数字识别不是从分类器开始, 而是从图像标准化开始。灰度化、阈值、反色、裁剪、缩放、展平这些步骤决定了模型真正看见的输入。

6.6 图像滤波

滤波的目标是改变像素与邻域之间的关系。平滑滤波可以降低噪声, 但也可能模糊边缘; 锐化和梯度滤波可以突出变化, 但也会放大噪声。因此滤波不是固定套路, 而是根据后续任务选择。

最一般的二维滤波可以写成卷积核作用在图像上。OpenCV 使用 `cv2.filter2D` 完成自定义核滤波。

Listing 228: 自定义卷积核滤波

```
1 kernel = np.ones((3, 3), np.float32) / 9
2 blur = cv2.filter2D(img_bgr, -1, kernel)
```

-1 表示输出图像使用和输入图像相同的深度。均值滤波把邻域像素取平均，可以压低随机噪声，但边界会变软。

Listing 229: 均值滤波与方框滤波

```
1 mean_blur = cv2.blur(img_bgr, (5, 5))
2 box_blur = cv2.boxFilter(img_bgr, -1, (5, 5), normalize=True)
```

高斯滤波使用越靠近中心权重越大的核，比简单均值更符合“近处像素更相关”的直觉，也常作为边缘检测前的去噪步骤。

Listing 230: 高斯滤波

```
1 gauss = cv2.GaussianBlur(img_bgr, (5, 5), 0)
```

中值滤波把邻域内像素排序后取中位数，对椒盐噪声尤其有效。它不是线性卷积，但能较好保留边缘。

Listing 231: 中值滤波

```
1 median = cv2.medianBlur(img_bgr, 5)
```

双边滤波同时考虑空间距离和颜色差异，距离近且颜色相近的像素才会互相影响，所以它常用于“保边去噪”。

Listing 232: 双边滤波

```
1 bilateral = cv2.bilateralFilter(img_bgr, d=9, sigmaColor=75,
    sigmaSpace=75)
```

滤波效果可以放在同一张图里比较：

Listing 233: 比较多种滤波结果

```
1 images = [img_bgr, mean_blur, gauss, median, bilateral]
2 titles = ["origin", "mean", "gaussian", "median", "bilateral"]
3
4 for i, (im, title) in enumerate(zip(images, titles), start=1):
5     plt.subplot(1, 5, i)
6     plt.imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
7     plt.title(title)
8     plt.axis("off")
9 plt.show()
```

核越大，平滑越强，细节损失也越明显。边缘检测之前常先做轻度高斯滤波；如果图像有明显椒盐噪声，中值滤波通常更合适。

6.7 图像边缘检测

边缘是灰度或颜色快速变化的位置。数学上，它和导数有关：一阶导数大的位置表示变化快；二阶导数过零或变化剧烈的位置也能反映边缘。OpenCV 中常见边缘算子包括 Sobel、Scharr、Laplacian 和 Canny。

Sobel 算子分别估计 x 方向和 y 方向的梯度：

Listing 234: Sobel 边缘

```
1 gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
2
3 sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
4 sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
5
6 sobel_x = cv2.convertScaleAbs(sobel_x)
7 sobel_y = cv2.convertScaleAbs(sobel_y)
8 sobel = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)
```

CV_64F 用来保留负梯度；如果直接使用无符号整数类型，负值会丢失，边缘结果会不完整。最后再用 `convertScaleAbs` 转成可显示的 8 位图像。

Scharr 可以看作 Sobel 在 3×3 尺寸下的改进版本，对小核梯度更敏感：

Listing 235: Scharr 边缘

```
1 scharr_x = cv2.Scharr(gray, cv2.CV_64F, 1, 0)
2 scharr_y = cv2.Scharr(gray, cv2.CV_64F, 0, 1)
3 scharr = cv2.addWeighted(
4     cv2.convertScaleAbs(scharr_x), 0.5,
5     cv2.convertScaleAbs(scharr_y), 0.5,
6     0
7 )
```

Laplacian 是二阶导数算子，不区分 x 和 y 方向，能突出灰度快速变化的位置。

Listing 236: Laplacian 边缘

```
1 lap = cv2.Laplacian(gray, cv2.CV_64F)
2 lap = cv2.convertScaleAbs(lap)
```

Canny 是更完整的边缘检测流程，内部包含去噪、梯度计算、非极大值抑制和双阈值连接。实际使用时，常先转灰度，再高斯滤波，最后 Canny。

Listing 237: Canny 边缘检测

```
1 gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
2 blur = cv2.GaussianBlur(gray, (5, 5), 0)
3 edges = cv2.Canny(blur, threshold1=50, threshold2=150)
4
5 plt.imshow(edges, cmap="gray")
6 plt.axis("off")
7 plt.show()
```


低阈值太小会保留大量噪声边缘，高阈值太大则会丢掉弱边缘。调参时应结合任务目标：人脸轮廓、道路车道线、证件文字边缘，需要的阈值范围可能完全不同。

6.8 实验：车道线滤波检测

车道线检测是把前面几个基础步骤串起来的案例。典型流程是：读入道路图像，转灰度，高斯滤波，Canny 边缘检测，选取道路区域作为感兴趣区域，用霍夫直线变换找线段，最后把线段画回原图。

Listing 238: 车道线检测的基本流程

```
1 img = cv2.imread("road.jpg")
2 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
3 blur = cv2.GaussianBlur(gray, (5, 5), 0)
4 edges = cv2.Canny(blur, 50, 150)
```

道路图像中天空、树木、车内仪表台等区域对车道线识别没有帮助，可以用多边形遮罩只保留下半部分道路区域。

Listing 239: 道路感兴趣区域遮罩

```
1 h, w = edges.shape
2 mask = np.zeros_like(edges)
3 polygon = np.array([[
4     (int(0.1 * w), h),
5     (int(0.45 * w), int(0.6 * h)),
6     (int(0.55 * w), int(0.6 * h)),
7     (int(0.95 * w), h),
8 ]], dtype=np.int32)
9
10 cv2.fillPoly(mask, polygon, 255)
11 roi_edges = cv2.bitwise_and(edges, mask)
```

霍夫直线变换可以从边缘图中找出近似直线段：

Listing 240: 概率霍夫直线变换

```
1 lines = cv2.HoughLinesP(
2     roi_edges,
3     rho=1,
4     theta=np.pi / 180,
5     threshold=30,
6     minLineLength=40,
7     maxLineGap=20
8 )
```

检测到线段后，可以画到空白图层上，再和原图叠加。

Listing 241: 绘制并叠加车道线

```
1 line_img = np.zeros_like(img)
2
```



```

3  if lines is not None:
4      for line in lines:
5          x1, y1, x2, y2 = line[0]
6          cv2.line(line_img, (x1, y1), (x2, y2), (0, 0, 255), 3)
7
8  result = cv2.addWeighted(img, 0.8, line_img, 1.0, 0)
9
10 plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
11 plt.axis("off")
12 plt.show()

```

这个实验的关键不是某个固定参数，而是理解每一步的输入输出：Canny 的输入是平滑后的灰度图，Hough 的输入是边缘二值图，绘图叠加的对象是原始彩色图。任何一步图像类型或尺寸不一致，都会导致结果错误。

6.9 图像几何变换

图像几何变换改变像素位置。常见操作包括缩放、翻转、平移、旋转、仿射变换和透视变换。它们处理的是坐标关系，而不是颜色或灰度本身。

缩放使用 `cv2.resize`。可以直接指定输出尺寸，也可以指定比例因子。

Listing 242: 图像缩放

```

1  small = cv2.resize(img_bgr, (200, 200))
2  half = cv2.resize(img_bgr, None, fx=0.5, fy=0.5, interpolation=cv2.
    INTER_AREA)
3  large = cv2.resize(img_bgr, None, fx=2.0, fy=2.0, interpolation=cv2.
    INTER_CUBIC)

```

缩小时常用 `INTER_AREA`，放大时常用 `INTER_LINEAR` 或 `INTER_CUBIC`。插值方法会影响图像边缘和细节。

翻转使用 `cv2.flip`：

Listing 243: 图像翻转

```

1  flip_x = cv2.flip(img_bgr, 0)      # 上下翻转
2  flip_y = cv2.flip(img_bgr, 1)      # 左右翻转
3  flip_xy = cv2.flip(img_bgr, -1)    # 上下和左右都翻转

```

平移可以写成仿射矩阵。矩阵中最后一列表示水平和垂直方向移动的像素数。

Listing 244: 图像平移

```

1  h, w = img_bgr.shape[:2]
2  M = np.float32([
3      [1, 0, 50],
4      [0, 1, 30],
5  ])
6  shifted = cv2.warpAffine(img_bgr, M, (w, h))

```

旋转也使用仿射变换。OpenCV 可以根据旋转中心、角度和缩放比例生成旋转矩阵。

Listing 245: 图像旋转

```
1 h, w = img_bgr.shape[:2]
2 center = (w // 2, h // 2)
3 M = cv2.getRotationMatrix2D(center, angle=45, scale=1.0)
4 rotated = cv2.warpAffine(img_bgr, M, (w, h))
```

一般仿射变换由三对点决定。仿射变换保持直线仍为直线，平行线仍为平行线，适合做平移、旋转、缩放、剪切等组合。

Listing 246: 三点确定仿射变换

```
1 h, w = img_bgr.shape[:2]
2 src = np.float32([[50, 50], [200, 50], [50, 200]])
3 dst = np.float32([[30, 80], [220, 40], [80, 230]])
4
5 M = cv2.getAffineTransform(src, dst)
6 affine = cv2.warpAffine(img_bgr, M, (w, h))
```

透视变换由四对点决定，可以处理拍摄角度导致的梯形变形。例如把倾斜拍摄的证件、标志牌或纸张校正成正面视角。

Listing 247: 四点透视校正

```
1 src = np.float32([
2     [120, 80],
3     [420, 60],
4     [450, 300],
5     [100, 320],
6 ])
7 dst = np.float32([
8     [0, 0],
9     [300, 0],
10    [300, 220],
11    [0, 220],
12 ])
13
14 M = cv2.getPerspectiveTransform(src, dst)
15 warped = cv2.warpPerspective(img_bgr, M, (300, 220))
```

几何变换会改变图像内容的位置和边界。变换后超出画布的部分会被裁掉，空出来的位置会用边界填充值补齐。实际项目中应先决定输出画布大小，再决定变换矩阵。

几何变换的坐标意识

颜色变换和滤波主要改变像素值，几何变换主要改变像素位置。写几何变换代码时，要同时关注输入图像尺寸、输出画布尺寸、坐标点顺序和插值方法。

6.10 复习与练习

练习 6.1. 读入一张彩色图，分别用 OpenCV 窗口和 Matplotlib 显示。故意不做 BGR 到 RGB 的转换，观察颜色错误，再修正。

练习 6.2. 读入一张图像，输出它的 `shape`、`dtype`、最大值、最小值，并分别裁剪出左上角、中心区域和右下角。

练习 6.3. 把一张彩色图转为灰度图，绘制灰度直方图。分别使用固定阈值和 Otsu 阈值二值化，比较两种结果。

练习 6.4. 找一张手写数字图片，完成灰度化、反向二值化、裁剪、缩放到 28×28 、展平成一维向量的流程。

练习 6.5. 在同一张含噪声图像上比较均值滤波、高斯滤波、中值滤波和双边滤波。说明哪一种更适合去椒盐噪声，哪一种更适合保留边缘。

练习 6.6. 分别使用 Sobel、Scharr、Laplacian 和 Canny 对同一张灰度图做边缘检测。调整 Canny 的两个阈值，观察弱边缘和噪声的变化。

练习 6.7. 用道路图片实现一次简化车道线检测：灰度化、高斯滤波、Canny、ROI 遮罩、霍夫直线检测、原图叠加。逐步显示每一步中间结果。

练习 6.8. 对一张图片分别做缩放、翻转、平移和旋转。记录每次输出图像的尺寸，解释为什么旋转后可能出现黑边或内容被裁掉。

练习 6.9. 选择一张倾斜拍摄的纸张或证件图，手动给出四个角点，用透视变换把它校正成矩形。

7 课程综合项目

前面各章分别学习了 Python、NumPy、SciPy、Matplotlib、Pandas 和 OpenCV。真正做科学计算时，它们通常不是分开出现的。一个完整任务往往从数据进入开始，经过清洗、数组计算、统计分析、建模或图像处理，最后用图形和文字说明结果。

7.1 项目一：表格数据分析报告

这个项目适合使用 CSV 或 Excel 数据，例如房价、成绩、销售、天气或乘客信息。目标不是把所有函数都用一遍，而是形成一条可复现的数据分析链路。

第一步是读入数据并检查结构：

Listing 248: 项目一的数据检查起点

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("data.csv")
5
6 print(df.shape)
7 print(df.columns)
8 print(df.dtypes)
9 print(df.head())
10 print(df.isnull().sum())
```

第二步是处理字段。数值列要确认单位和类型，类别列要检查空格、大小写和缺失值，日期列要转换成日期类型。

Listing 249: 字段清洗示例

```
1 df["city"] = df["city"].str.strip().str.upper()
2 df["date"] = pd.to_datetime(df["date"])
3 df["month"] = df["date"].dt.month
4 df["value"] = pd.to_numeric(df["value"], errors="coerce")
5
6 df = df.dropna(subset=["city", "month", "value"])
```

第三步是提出可计算的问题。比如“不同城市每月平均值是否不同”，可以先分组聚合，再透视成宽表。

Listing 250: 分组、透视和可视化

```
1 monthly = df.groupby(["city", "month"])["value"].mean().reset_index()
2
3 table = monthly.pivot_table(
4     index="city",
5     columns="month",
6     values="value",
7     aggfunc="mean"
```

```

8 )
9
10 ax = table.T.plot(marker="o")
11 ax.set_xlabel("month")
12 ax.set_ylabel("mean value")
13 ax.legend(title="city")
14 plt.tight_layout()
15 plt.show()

```

如果要写成报告，至少应说明数据来源、样本量、清洗规则、分组方式和图形结论。结论不能只写“图像如上”，而要指出具体差异来自哪一列、哪一组、哪一段时间。

表格项目的最低交付标准

报告中必须能回答四个问题：数据有多少行多少列，哪些字段被清洗或删除，核心统计量怎样计算，图形支持了什么结论。任何一步说不清，后面的图都不可靠。

7.2 项目二：带噪声信号的频域分析

这个项目把 NumPy、SciPy 和 Matplotlib 连起来。任务是构造一个含多个频率的信号，加入噪声，再比较时域滤波和频域滤波。

Listing 251: 构造信号并加入噪声

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal, fftpack
4
5 np.random.seed(0)
6
7 fs = 100
8 t = np.arange(0, 5, 1 / fs)
9 clean = np.sin(2 * np.pi * 3 * t) + 0.5 * np.sin(2 * np.pi * 12 * t)
10 raw = clean + np.random.normal(0, 0.4, t.shape)

```

时域滤波可以先用中值滤波或 Wiener 滤波观察效果：

Listing 252: 时域滤波对比

```

1 median = signal.medfilt(raw, kernel_size=5)
2 wiener = signal.wiener(raw)
3
4 plt.plot(t, raw, alpha=0.4, label="raw")
5 plt.plot(t, median, label="median")
6 plt.plot(t, wiener, label="wiener")
7 plt.legend()
8 plt.show()

```

频域分析要同时看频率坐标和幅度：

Listing 253: 频谱分析和低通滤波

```

1 F = fftpack.fft(raw)
2 freq = fftpack.fftfreq(len(raw), d=1 / fs)
3
4 mask = freq > 0
5 plt.plot(freq[mask], np.abs(F)[mask])
6 plt.xlabel("frequency")
7 plt.ylabel("amplitude")
8 plt.show()
9
10 F_filtered = F.copy()
11 F_filtered[np.abs(freq) > 15] = 0
12 freq_filtered = fftpack.ifft(F_filtered).real

```

最后把原信号、含噪信号、滤波后信号放在同一张图中。若滤波阈值选择为 15 Hz，应解释它和信号中 3 Hz、12 Hz 成分之间的关系，而不是只说“效果更平滑”。

Listing 254: 最终结果对比

```

1 plt.plot(t, clean, label="clean")
2 plt.plot(t, raw, alpha=0.3, label="raw")
3 plt.plot(t, freq_filtered, label="fft filtered")
4 plt.legend()
5 plt.show()

```

7.3 项目三：图像目标提取

这个项目适合选择一张目标明确的图像，例如手写数字、标志牌、证件、车道线或简单物体。任务不是直接调用一个高级模型，而是用 OpenCV 把目标区域一步步提取出来。

基本流程从读图和颜色转换开始：

Listing 255: 图像项目起点

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 img = cv2.imread("target.jpg")
6 if img is None:
7     raise FileNotFoundError("target.jpg")
8
9 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
10 blur = cv2.GaussianBlur(gray, (5, 5), 0)

```

若目标和背景在亮度上能分开，可以先用 Otsu 阈值；若目标主要体现为边界，可以使用 Canny。

Listing 256: 两种目标提取入口

```

1 _, binary = cv2.threshold(

```

```

2     blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
3 )
4
5 edges = cv2.Canny(blur, 50, 150)

```

对于块状目标，继续找轮廓和外接框：

Listing 257: 轮廓筛选目标区域

```

1 contours, hierarchy = cv2.findContours(
2     binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
3 )
4
5 vis = img.copy()
6 for cnt in contours:
7     area = cv2.contourArea(cnt)
8     if area < 100:
9         continue
10    x, y, w, h = cv2.boundingRect(cnt)
11    cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 0, 255), 2)
12
13 plt.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))
14 plt.axis("off")
15 plt.show()

```

对于线状目标，例如车道线，可以用 Canny 结果接霍夫直线。对于倾斜矩形目标，例如证件和纸张，可以用四点透视变换校正。不同目标的后半段不同，但前半段都要求看清图像数组、灰度图、二值图或边缘图。

图像项目的检查图

图像项目至少保存四张检查图：原图、灰度或滤波图、二值图或边缘图、最终标注图。只给最终结果看不出错误来自哪里，也无法判断阈值和轮廓筛选是否合理。

7.4 综合项目复习要求

练习 7.1. 选择一份表格数据，完成读入、字段检查、缺失值处理、分组聚合、透视表和图形导出。报告中写清楚每一步删掉或改变了哪些数据。

练习 7.2. 构造一个由两个正弦波叠加而成的信号，加入随机噪声，分别使用时域滤波和频域滤波处理。要求解释采样率、频率峰和滤波阈值之间的关系。

练习 7.3. 选择一张图片，用 OpenCV 完成目标提取。要求输出原图、预处理图、中间二值图或边缘图、最终目标标注图，并说明每个参数为什么这样选。

练习 7.4. 把任意一个综合项目整理成可复现 Notebook。重启内核后从头运行，确保所有图形和结果都能重新生成。